About MongoDB

MongoDB is a document-oriented, operational database built from the ground up as an alternative to the relational database for modern applications. Unlike relational databases, MongoDB allows developers to store rich JSON-like documents that map naturally to the objects they use in their code:

MongoDB : Notes Index

MongoDB : Notes Index

What is : MongoDB

MongoDB is a **NoSQL database** that stores data not in tables and rows,
but in **documents** (JSON-like format). It is an open-source, cross-platform database that is highly flexible and scalable.
Unlike traditional SQL databases where you must define a schema first,
MongoDB has no such restriction — you can change your data structure anytime. Its name comes from "Humongous" — meaning very large — because it was built to handle massive amounts of data

Key Points

Data is stored in **documents** (BSON format — Binary JSON)
Documents live inside **collections** (similar to SQL tables)
Schema-less — each document can have a different structure
Very **fast read/write** — especially for unstructured data
Horizontally scalable — add more servers as data grows
Powerful **query language** — filter, sort, aggregate all supported
MongoDB Atlas — cloud version available (free tier included)

Use Case	Example
E-commerce	Products, orders, user profiles
Mobile Apps	User data, notifications, activity logs
Content Management	Blogs, articles, media metadata
Gaming	Player stats, leaderboards, game state
Real-time Apps	Chat apps, live dashboards
Big Data	Logs, analytics, IoT sensor data

Step 1 : Download MongoDB Server

- Version: 8.0.9 (Current)
- Platform: Windows
- Package: MSI
- During install → keep "Install MongoDB as a Service" ✓ checked

1 https://www.mongodb.com/try/download/community

Step 2 : Download mongosh (MongoDB Shell)

Mongosh is installed separately from the server, Download → Install → Path is set automatically

2 https://www.mongodb.com/try/download/shell

Step 3 : Verify Installation

Open CMD (command prompt) and run this command to verify the installation setup

3 mongosh

If everything is set up correctly, you will see:

```
C:\Users\Hp>mongosh
Current Mongosh Log ID: 6a130e52acaa0005396c4bcf
Connecting to:    mongodb://127.0.0.1:27017/?directConnection=true
Using MongoDB:    8.0.9
Using Mongosh:    2.5.1

test>
```

Database : Show

This command show all database (with name and size in mongosh shell command tool)

```
1  show dbs
```

Database : Create

This command use create new database or inter if database already exist, use to switch database

```
2  use school
```

Database : Current

This command show currently you in which database, return name of database

```
3  db
```

Database : Delete

This command delete database currently you use, and return response like : { ok: 1, dropped: 'school' }

```
4  db.dropDatabase()
```

Command : Help

This command show all command related to db

```
5  db.help()
```

Collection : Create

This command create new collection name students return response

```
1  db.createCollection("students")
```

Collection : Show

This command return all collection in database

```
2  show collections
```

Collection : Rename

This command rename the collection and return response {ok:1}, example : db.old_name.renameCollection("new_name")

```
3  db.students.renameCollection("student")
```

Collection : Delete

This command delete the collection return response { ok: 1 }, example : db.collectionName.drop()

```
4  db.student.drop()
```

Collection : Help

This command show all command related to collection example db.collectionName.help()

```
5  db.student.help()
```

Insert : One

It inserts a new document into the students collection, name: "Anjesh Kumar" age: 25 class: "None"

```
1  db.students.insertOne({ name: "Anjesh Kumar", age: 25, class: "None" });
```

Insert : Many

It inserts multiple documents (3 in this case) into the students collection at once.
Each document represents a student with name, age, and class fields
Each will get a unique _id automatically generated by MongoDB.

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 25, class: "None" },
  { name: "Ankit  Kumar", age: 25, class: "None" },
  { name: "Aniket Kumar", age: 10, class: "None" },
]);
```

Data : Types

This is all available data types in MongoDB

1 **MongoDB Data Types Listed:**

- String
- Double
- 32-bit Integer (int)
- 64-bit Integer (int)
- Boolean (bool)
- Array
- Object
- Null
- Regular Expression (highlighted in red)
- Timestamp (highlighted in red)
- Date
- ObjectId

```
{
  name: "Yahoo Baba",
  age: 25,
  married: false,
  dob: ISODate("2000-10-15T08:00:00Z"),
  weight: 72.50,
  kids: null,
  hobbies: ["music", "sports"],
  address: {
    street: "123 Main St",
    city: "New York",
    zip: 10001
  }
}
```

32 bit : Integer

Uses 32 bits (4 bytes) of memory to store the integer value.

2 `-2,147,483,648 to 2,147,483,647`

64 bit : Integer

Uses 64 bits (8 bytes) of memory to store the integer value.

3 `-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807`

Date : ISODate

This method use for save date and time, z stand for Coordinate Universal Time, without z then its will a local time zone

4 `{dob: ISODate("2000-10-15T08:00:00Z")}`

Date : new Date

This is second method use for save date and time, z stand for Coordinate Universal Time, without z then its will a local time zone

5 `{dob: new Date("2000-10-15T08:00:00Z")}`

Date : Current Date

This method save current time

6 `{ date: new Date() }`

Validation Shema : For existing collection

If you want to add validation schema in existing collection follow this complete example (make collection name personal)

```
db.runCommand({
  collMod: "personal",        //collection name which one you want add validation
  validator: {
    $jsonSchema: {
      required: ["name", "age", "married", "dob", "weight", "kids", "address"],
      properties: { //add property's and validation rule & fails message
        name: {
          bsonType: "string",            //string type
          description: "Name must be a string and is required", // fails message
        },
        age: {
          bsonType: "int",               //integer type
          minimum: 20,
          maximum: 35,
          description: "Age must be an integer between 20 and 35 is required",
        },
        married: {
          bsonType: "bool",    //Boolean type
          description: "Married must be boolean (trune/false)", // fails message
        },
        dob: {
          bsonType: "date",    //date type
          description: "DOB must be an date", // fails message
        },
        weight: {
          bsonType: "double",  //double or float type
          description: "Weight must be an duble.", // fails message
        },
        kids: {
          bsonType: "int",               //integer type
          description: "Kids must be an integer and is required", // fails message
        },
        hobbies: {
          bsonType: "array",   //array type
          items: {
            bsonType: "string",          //array items data types
          },
          description: "Hobbies must be an array of strings ", // fails message
        },
        address: {
          bsonType: "object", //object type
          required: ["street", "city", "zip"], //object require property's
          properties: {
            street: {                    //object property data type
              bsonType: "string",
              description: "Street must be a string and is required", // fails message
            },
            city: {
              bsonType: "string",
              description: "City must be a string and is required", // fails message
            },
            zip: {
              bsonType: "int",
              description: "Zip must be a int and is required", // fails message
            },
          }, }, }, }, }, }); // fails message show when validation fails
```

Validation Schema : For New Collection

Create new collection and define validation schema

```
db.createCollection("students", {

  // Main validator object
  validator: {
    $jsonSchema: {
      title: "Student Object validation",  // Title show when validation fails
      required: ["name", "age", "course"], // Define all require field here

     // Define validation rule for all field in properties object
      properties: {
        name: {
          bsonType: "string", // define type in bsonType properties
          description: "must be a string and is required",
        }, // msg show when validation false (description show when validation false
        age: {
          bsonType: "int", // field type int = integer
          minimum: 5,
          maximum: 20,
          description: "age must be an integer in [5, 20] and is required",
        },
        course: {
          bsonType: "string",
          enum: ["BCA", "BBA", "BSc", "MCA", "MBA"], // pre-define values in enum array
          description: "course must be a string and is required",
        },
      },},},});
```

Insert : Data

Let's insert data in student collection with correct data type, but when you pass wrong datatype in any field that through a error

```
db.students.insertOne({
  name: "Anjesh Kumar",
  age: 19,
  course: "BCA",
});
```

Insert : Data

Let's insert data in personal collection with correct data type, but when you pass wrong datatype in any field that through a error

```
db.personal.insertOne({
  name: 22,
  age: 30,
  married: true,
  dob: new Date("1993-05-15"),
  weight: 70.5,
  kids: 2,
  hobbies: ["reading", "traveling"],
  address: {
    street: "123 Main St",
    city: "Delhi",
    zip: 12345,
  },
});
```

Update Document : updateOne,updateMany

Collection : For Testing Query's

Create a Collection name students for testing updateOne, updateMany Method & Operators

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Btech", skills: ["HTML", "PHP"] },
  { name: "Ashito Kumar", age: 20, class: "Btech", skills: ["React", "Js"] },
  { name: "Aniket Kumar", age: 17, class: "Mtech", skills: ["Mongo", "SQL"] },
]);
```

Operator : $set

Searches for the first document where name is "Anjesh Kumar", Updates the name to "Anjesh" and age to 17

```
db.students.updateOne(
  { name: "Anjesh Kumar" }, //find the document
  {
    $set: {
      age: 17,
      name: "Anjesh", //update new value
    },
  }
);
```

Operator : $set

Finds the document with the exact _id, Updates (or adds) the field class and sets it to "M.tech"

```
db.students.updateOne(
  { _id: ObjectId("68410ffa1c5b77ca826c4be6") }, // find by id
  {
    $set: {
      class: "M.tech", //update new value to class field
    },
  }
);
```

Operator : $inc

Finds the first document where name is "Aniket Kumar", Increases the value of the age field by 1

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $inc: {
      age: 1, // Increment the age of the student by 1
    },
  }
);
```

Operator : $inc

Finds the first document where name is "Aniket Kumar", Decrements the age field by 1 using $inc: { age: -1 }

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $inc: {
      age: -1, // Decrements the age of the student by 1
    },
  }
);
```

Operator : $mul

Finds the first document where name is "Anjesh", Multiplies the age field by 2 using the $mul operator

```
db.students.updateOne(
  { name: "Anjesh" },
  {
    $mul: {
      age: 2, // Multiply the age of the student by 2
    },
  }
);
```

Operator : $unset

Remove the field using $unset operator

```
db.students.updateOne(
  { name: "Anjesh" },
  {
    $unset: {
      age: "", // Remove the age field from the document
    },
  }
);
```

Operator : $rename

Rename the field name using $rename operator (using updateOne you can rename only one document)

```
db.students.updateOne(
  { name: "Anjesh" },
  {
    $rename: {
      skills: "coding_skills", // Rename the skills field to coding_skills
    },
  }
);
```

Operator : $rename

Rename the field name for all document in collection using updateMany for update all document in collection

```
db.students.updateMany( {}, //empty object for run operator for all document
  {
    $rename: {
      skills: "coding_skills", // Rename the skills field to coding_skills
    },
  }
);
```

Operator : $currentDate

Set the current date to all field, if field not exit then this will create the field and add current date, using updateMany

```
db.students.updateMany( {}, //empty object for run operator for all document
  {
    $currentDate: {
      lastModified: true, //field name and value true for add current date
    },
  }
);
```

Update Document : updateOne,updateMany - III

Operator : $min

Update the minimum value to field (if field value less then update value then that value update to document

```
db.students.updateOne(
  { name: "Ashito Kumar" },
  {
    $min: {
      age: 18, // Set the minimum age to 18, if the current age is less than 18
    },
  }
);
```

Operator : $max

Update the Max value to field (if field value greater then update value then that value update to document

```
db.students.updateOne(
  { name: "Ashito Kumar" },
  {
    $max: {
      age: 25, // Set the maximum age to 25, if the current age is greater than 25
    },
  }
);
```

Array Operator : $push

Finds the document where name is "Aniket Kumar", Adds "Python" to the coding_skills array using $push.

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $push: {
      coding_skills: "Python", // Add "Python" to the coding_skills array
    },
  }
);
```

Array Operator : $addToSet

Finds the student where name is "Aniket Kumar", Adds "Java" to the coding_skills array only if it's not already present.

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $addToSet: {
      coding_skills: "Java", // Add "Java" to the coding_skills array only if it does not already exist
    },
  });
```

Array Operator : $pop

This query will remove the last element from the coding_skills array for the document where the name is "Aniket Kumar".

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $pop: {
      coding_skills: 1, // Set 1 for Remove the last value from the coding_skills
    },                                                                    array
  });
```

Array Operator : $pop

This query will remove the first element from the coding_skills array for the document where the name is "Aniket Kumar".

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $pop: {
      coding_skills: -1, // Remove the first value from the coding_skills array
    },
  }
);
```

Array Operator : $pull

Using this operator you can remove specific value from array (but you can only one value find and remove)

```
db.students.updateOne(
  { name: "Ashito Kumar" },
  {
    $pull: {
      coding_skills: "Js", // Remove "Js" from the coding_skills array
    },
  }
);
```

Array Operator : $pullAll

Using this operator you can remove multiple specific value form array

```
db.students.updateOne(
  { name: "Aniket Kumar" },
  {
    $pullAll: {
      coding_skills: ["SQL", "Python"], // Remove multiple value from the
    },                                                    coding_skills array
  }
);
```

Method : replaceOne

Using this Method You can find and replace document values

```
db.students.replaceOne(
  {
    _id: ObjectId("684114511c5b77ca826c4be7"), //Replace the document with specific
  },                                                                    object id
  {
    name: "Anjesh Kumar",
    age: 18,
    class: "Btech",
    coding_skills: ["Mongo", "SQL", "Python"],
  }
);
```

Update Document : deleteOne, deleteMany

Collection : For Testing Query's

Create a Collection name students for testing deleteOne, deleteMany Method

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Btech", skills: ["HTML", "PHP"] },
  { name: "Ashito Kumar", age: 20, class: "Btech", skills: ["React", "Js"] },
  { name: "Aniket Kumar", age: 17, class: "Mtech", skills: ["Mongo", "SQL"] },
]);
```

Method : deleteOne

deleteOne() removes only the first matching document based on the given filter.

```
db.students.deleteOne({
  name: "Anjesh Kumar", // Delete the first document with name "Anjesh Kumar"
});

db.students.deleteOne({
  age: 17,                // Delete the first document with age 17
});
```

Method : deleteOne

Deletes one document from the students collection whose _id exactly matches

```
db.students.deleteOne({
  _id: ObjectId("684114511c5b77ca826c4be8"), //Delete the document with the by ObjectId
});
```

Method : deleteMany

This query deletes all documents from the students collection where the class field is exactly "Btech".

```
db.students.deleteMany({
  class: "Btech",   // Delete all documents with class "Btech"
});
```

Method : deleteMany

This deletes all documents from the students collection — entire data wipe, but does not delete the collection itself.

```
db.students.deleteMany({}); // Delete all documents in the collection
```

Read Document : find, findOne

Collection : For Testing Query's

Create a Collection name students for testing find(), findOne() method

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Mtech", city: "Delhi" },
  { name: "Ashito Kumar", age: 22, class: "Btech", city: "Mumbai" },
  { name: "Aniket Kumar", age: 20, class: "Btech", city: "Delhi" },
  { name: "Zubair Ahmad", age: 18, class: "Mtech", city: "Kolkata" },
  { name: "Rajvir Kumar", age: 11, class: "Btech", city: "Bihar" },
]);
```

Read Data : find

Each .find() returns a cursor (list of matching documents) from the students collection in the current MongoDB database.

```
db.students.find({
  class: "Btech",                    // find all students where class = Btech
});

db.students.find({
  age: 20,                           // find all students where age = 20
});

db.students.find({
  city: "Delhi",                     // find all students where city = Delhi
});
```

Read Data : findOne

This code is using db.students.findOne() — which returns only the first matching document from the students collection.

```
db.students.findOne({
  city: "Delhi",                     // find first one student in Delhi
});

db.students.findOne({
  class: "Btech",                    // find first one student in Btech
});
```

Read Data : findOne with projection

It returns all students, but only shows name and age for each — no _id

```
db.students.find().projection({
  name: 1,                           // projection to include name
  age: 1,
  _id: 0,
});
```

Read Data : findOne with projection

Finds all students in class "Btech" Displays only name and age Hides the _id field (which is included by default)

```
db.students.find({
    class: "Btech",                  // find all students in Btech
  },{
    name: 1,                         // projection to include name
    age: 1,
    _id: 0,
  });
```

Method : count

This returns the total number of documents (students) in the students collection.

```
db.students.find().count();             // Return total count of all students
```

Count : with filter

This returns the total number of students in class "Btech".

```
db.students.find({
    class: "Btech",                     // Return total count of all students in Btec
  }).count();
```

Read data in ascending Order : sort

Returns all students Sorted by the city field in ascending (A–Z) order

```
db.students.find().sort({
  city: 1,                              // sort by city in ascending order
});
```

Read data in descending : sort

Returns all student documents Sorts them by age in descending order (from oldest to youngest)

```
db.students.find().sort({
  age: -1,                              // sort by age in descending order
});
```

Read data in ascending order with filter : sort

Returns all students Shows only their name and city Sorts them by age from youngest to oldest

```
db.students.find({}, { name: 1, city: 1, _id: 0 }).sort({
  age: 1,                               // sort by age in ascending order
});
```

Read limited data : limit

Returns only the first 2 documents from the students collection Based on the default insertion order, unless combined with .sort()

```
db.students.find().limit(2);            // Limit the result to 2 documents
```

Read limited data : limit

limit(n) to restrict how many results are returned .skip(n) to skip a certain number of documents Together, they enable pagination

```
db.students.find().limit(2).skip(0);
// Skip the first 0 documents and limit the result to 2 documents

db.students.find().limit(2).skip(2);
// Skip the first 2 documents and limit the result to 2 documents

db.students.find().limit(2).skip(4);
// Skip the first 4 documents and limit the result to 2 documents
```

Collection : For Testing Query

Create a Collection name students for testing Comparison Operators

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Mtech", city: "Delhi" },
  { name: "Ashito Kumar", age: 22, class: "Btech", city: "Mumbai" },
  { name: "Aniket Kumar", age: 20, class: "Btech", city: "Delhi" },
  { name: "Zubair Ahmad", age: 18, class: "Mtech", city: "Kolkata" },
  { name: "Rajvir Kumar", age: 11, class: "Btech", city: "Bihar" },
]);
```

Equal : $eq

This query finds all documents in the students collection where the age field is exactly equal to 20.

```
db.students.find({
  age: { $eq: 20 },                    //Values are equal
});
```

Shorthand of : $eq

You can also write this more simply, Both are functionally the same

```
db.students.find({ age: 20 });         //Values are equal
```

Not Equal : $ne

This query will return all documents in the students collection where the age is not equal to 20.

```
db.students.find({
  age: { $ne: 20 },                    // Values are not equal
});
```

Greater Than : $gt

This query fetches all documents from the students collection where the age is greater than 20. (work only with numeric field)

```
db.students.find({
  age: { $gt: 20 },                    // Value is greater than
});
```

Greater Than or equal : $gte

This will return all documents in the students collection where age is greater than or equal to 20. (work only with numeric field)

```
db.students.find({
  age: { $gte: 20 },                   // Value is greater than or equal t
});
```

Less than : $lt

This query returns all documents in the students collection where age is less than 20. (work only with numeric field)

```
db.students.find({
  age: { $lt: 20 },                    // value is less than another value
});
```

Less than or equal : $lte

This retrieves all documents in the students collection where the age is less than or equal to 20. (work only with numeric field)

```
db.students.find({
  age: { $lte: 20 },                   // values is less than or equal to another value
});
```

In Array : $in

This query returns all documents from the students collection where the age is either 20, 18, or 17.

```
db.students.find({
  age: { $in: [20, 18, 17] },          // Value is matched within an array
});
```

Not In Array : $in

This query returns all documents in the students collection where the age is not 20, 18, or 17.

```
db.students.find({
  age: { $nin: [20, 18, 17] },         // value is not in the array
})
```

Example in updateMany method : $lte

all student documents where age < 20, those documents by setting a new (or replacing existing) field class with "12th"

```
db.students.updateMany(
  {
    age: { $lt: 20 }, // Update all students with age less than 20
  },
  {
    $set: {
      class: "12th", // Set class to "12th" for students with age less than 20
    },
  }
);
```

Example in deleteMany : $lte

This query deletes all student documents from the students collection where the age is less than 20.

```
db.students.deleteMany({
  age: { $lt: 20 }, // Delete all students with age less than 20
});
```

Collection : For Testing Query

Create a Collection name students for testing Logic Operators

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Mtech", city: "Delhi" },
  { name: "Ashito Kumar", age: 22, class: "Btech", city: "Mumbai" },
  { name: "Aniket Kumar", age: 20, class: "Btech", city: "Delhi" },
  { name: "Zubair Ahmad", age: 18, class: "Mtech", city: "Kolkata" },
  { name: "Rajvir Kumar", age: 11, class: "Btech", city: "Bihar" },
]);
```

Operator : $and

Returns documents where all conditions matches

```
db.students.find({
  $and: [{ age: { $gt: 20 } }, { age: { $lt: 23 } }],
}); // Find students with age greater than 20 and less than 23

db.students.find({
  $and: [{ age: { $gt: 20 } }, { city: { $eq: "Delhi" } }],
});// Find students with age greater than 20 and city equal to Delhi
```

Operator : $or

Returns documents where all or any single condition matches

```
db.students.find({
  $or: [{ age: { $gt: 20 } }, { city: { $eq: "Delhi" } }],
});// Find students with age greater than 20 or city equal to Delhi
```

Operator : $nor

Returns documents where both conditions fails or not matches

```
db.students.find({
  $nor: [{ age: { $gt: 20 } }, { city: { $eq: "Delhi" } }],
}); // Find students where age is not greater than 20 and city is not equal to Delhi
```

Operator : $not

Returns documents where the condition does not match

```
db.students.find({
  age: { $not: { $gt: 20 } },
});// Find students where age is not greater than 20
```

$and : with deleteMany

Returns documents where the condition does not match

```
db.students.deleteMany({
  $and: [{ age: { $gt: 20 } }, { city: { $eq: "Delhi" } }],
});// Delete students with age greater than 20 and city equal to Delhi
```

Collection : For Testing Query

Create a Collection name students for testing Element Operators

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Mtech", city: "Delhi" },
  { name: "Ashito Kumar", age: 22.5, class: "Btech", city: "Mumbai" },
  { name: "Aniket Kumar", age: "20", class: "Btech", city: "Delhi" },
  { name: "Zubair Ahmad", age: 18, city: "Kolkata" },
  { name: "Rajvir Kumar", age: 11, city: "Bihar" },
]);
```

Operator : $exists

Returns documents where the field exists or not

```
db.students.find({
  class: { $exists: true },
}); // Find students where class field exists
```

Operator : $type

Returns documents where the field is of a specific type (example for string type)

```
db.students.find({
  age: { $type: "string" },
}); // Find students where age field is of type string
```

Operator : $type

Returns documents where the field is of a specific type (example for Integer type)

```
db.students.find({
  age: { $type: "int" },
}); // Find students where age field is of type int
```

Operator : $type

Returns documents where the field is of a specific type (example for double type)

```
db.students.find({
  age: { $type: "double" },
}); // Find students where age field is of type double
```

Operator : $type

Returns documents where the field is of a specific type (example for match multiple data types)

```
db.students.find({
  age: { $type: ["int", "double"] },
}); // Find students where age field is of type int or double
```

Collection : For Testing Query

Create a Collection name students for testing Evaluation Operators

```
db.students.insertMany([
  { name: "Anjesh Kumar", age: 22, class: "Mtech", city: "Delhi" },
  { name: "Ashito Kumar", age: 22, class: "Btech", city: "Mumbai" },
  { name: "Aniket Kumar", age: 20, class: "Btech", city: "Delhi" },
  { name: "Zubair Ahmad", age: 18, class: "Mtech", city: "Kolkata" },
  { name: "Rajvir Kumar", age: 11, class: "Btech", city: "Bihar" },
]);
```

Operator : $regex

Returns documents where the field matches the regex pattern

```
db.students.find({
  name: { $regex: "Kumar" },   // Find students where name contains "Kumar"
});
```

Operator : $regex

Returns documents where the field matches the regex pattern

```
db.students.find({
  name: { $regex: /kumar/i }, // Find students where name contains "kumar" case
});                                                                  insensitive
```

Operator : $regex

Returns documents where the field matches the regex pattern

```
db.students.find({
  name: { $regex: "An" }, // Find students where name starts with "An"
});
```

Operator : $regex

Returns documents where the field matches the regex pattern

```
db.students.find({
  name: { $regex: "^A" }, // Find students where name starts with "A"
});
```

Operator : $regex

Returns documents where the field matches the regex pattern

```
db.students.find({
  name: { $regex: "ad$" }, // Find students where name ends with "ad"
});
```

Collection : For Testing Query

Create a Collection name monthlyBudget & sales for testing Evaluation Operators

```
db.monthlyBudget.insertMany([
  { _id: 1, category: "food", budget: 400, spent: 450 },
  { _id: 2, category: "drink", budget: 100, spent: 150 },
  { _id: 3, category: "clothes", budget: 100, spent: 50 },
  { _id: 4, category: "misc", budget: 500, spent: 300 },
  { _id: 5, category: "travel", budget: 200, spent: 650 },
]);

db.sales.insertMany([
  { _id: 1, product: "apple iPhone 13", cost: 800, price: 1000 },
  { _id: 2, product: "Samsung Galaxy S21", cost: 700, price: 950 },
  { _id: 3, product: "Apple Watch", cost: 300, price: 350 },
]);
```

Operator : $expr

Allows field comparison using within document

```
// Find documents where spent is greater than budget

db.monthlyBudget.find({
  $expr: { $gt: ["$spent", "$budget"] },
});
```

Operator : $expr

Allows field comparison using within document

```
// Find documents where price is greater than cost multiplied by 1.2

db.sales.find({
  $expr: { $gt: ["$price", { $multiply: ["$cost", 1.2] }] },
});
```

Operator : $mod

Matches numbers base on remainder

```
db.sales.find({
  cost: { $mod: [7, 0] }, // Find documents where cost is divisible by 7
});
```

Operator : $mod

Matches numbers base on remainder

```
db.sales.find({
  cost: { $mod: [7.0, 0] }, // Find documents where cost is divisible by 7.0
});
```

Operator : $mod

Matches numbers base on remainder

```
db.sales.find({
  cost: { $mod: [-7, -0] }, // Find documents where cost is divisible by -7
});
```

Collection : For Testing Query

Create a Collection name students for testing findOneAndUpdate, findOneAndDelete, findOneAndReplace

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", age: 23, class: "BCA" },
  { _id: 2, name: "Salman Khan", age: 24, class: "Btech" },
  { _id: 3, name: "Shahid Kapoor", age: 20, class: "BSc" },
  { _id: 4, name: "John Abraham", age: 19, class: "BCA" },
  { _id: 5, name: "Amir Khan", age: 24, class: "Btech" },
  { _id: 6, name: "Salman Khan", age: 21, class: "BSc" },
  { _id: 7, name: "Suniel Shetty", age: 22, class: "BCA" },
  { _id: 8, name: "Kartik Aryan", age: 20, class: "Btech" },
]);
```

Find and Update : findOneAndUpdate

Return inundate document and update the document

```
db.students.findOneAndUpdate(
{ name: "Kartik Aryan" },
{ $set: { age: 30 } }
);
```

Update With : returnDocument

Update the Document and Return update document

```
db.students.findOneAndUpdate(
  { name: "Kartik Aryan" },
  { $set: { age: 20 } },
  { returnDocument: "after" }          // return updated document
);
```

Update With : projection

Update the Document and Return update document with projection (which field you want to see)

```
db.students.findOneAndUpdate(
  { name: "Kartik Aryan" },              // filter
  { $set: { age: 30 } },                 // update document
  {
    returnDocument: "after",             // return updated document
    projection: { name: 1, age: 1, _id: 0 }, // projection
  }
);
```

Update : findOneAndUpdate

When filter not find any matched then return null

```
db.students.findOneAndUpdate(
  { name: "Anjeeeesh Kumar" },           // filter
  { $set: { age: 30 } },                 // update document
  { returnDocument: "after" }            // return updated document
);
```

Update With : upsert

When filter not find any matched then add new document and return the new document

```
db.students.findOneAndUpdate(
  { name: "Anjesh Kumar" }, // filter
  { $set: { age: 30 } }, // update document
  {
    returnDocument: "after", // return updated document
    projection: { name: 1, age: 1, _id: 0 }, // projection
    upsert: true, // if filter not find any matched then add new document
  }
);
```

Update With : short

Sort the document ascending order before update, update the first document and return the updated document

```
db.students.findOneAndUpdate(
  { name: "Salman Khan" },
  { $set: { class: "BTT" } },
  {
    sort: { age: 1 }, // sort by age in ascending order
  }
);
```

Replace : findOneAndReplace

Replace the document first matched and return the updated document

```
db.students.findOneAndReplace(
  { name: "Akshay Kumar" }, // filter
  { name: "Sanjay Dutt", age: 35, class: "BCom" },
  { returnDocument: "after" } // return updated document
);
```

Delete : findOneAndDelete

First sort the document by age in ascending order, delete the first matched document and return the deleted document

```
db.students.findOneAndDelete(
  { name: "Salman Khan" },
  {
    projection: { name: 1, age: 1, _id: 0 }, // projection
    sort: { age: 1 }, // sort by age in ascending order
  }
);
```

Collection : For Testing Query

Create a Collection name students for test Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", age: 23, class: "BCA" },
  { _id: 2, name: "Salman Khan", age: 24, class: "Btech" },
  { _id: 3, name: "Shahid Kapoor", age: 20, class: "BSc" },
  { _id: 4, name: "John Abraham", age: 19, class: "BCA" },
  { _id: 5, name: "Amir Khan", age: 24, class: "Btech" },
  { _id: 6, name: "Salman Khan", age: 21, class: "BSc" },
  { _id: 7, name: "Suniel Shetty", age: 22, class: "BCA" },
  { _id: 8, name: "Kartik Aryan", age: 20, class: "Btech" },
]);
```

Aggregate With : $match

Filter the documents based on the condition

```
// Match all students in BCA class
db.students.aggregate([{ $match: { class: "BCA" } }]);

// Match all students with age greater than 20
db.students.aggregate([{ $match: { age: { $gt: 20 } } }]);
```

Aggregate With : $match + $and

Match all students with age greater than 20 and class BCA

```
db.students.aggregate([
  {
    $match: {
      $and: [{ age: { $gt: 20 } }, { class: "BCA" }], // Using $and operator
    },
  },
]);
```

Aggregate With : $count

Count the number of documents that match the condition,

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },
  { $count: "name" }
]);
```

Aggregate With : $sortBycount

Count the number of documents for each unique value of a field returning the field and the count

```
db.students.aggregate([
   { $sortByCount: "$class" }
]);
```

Aggregate With : $sample

Randomly select a specified number of documents from the collection

```
db.students.aggregate([
  { $sample: { size: 2 } } // Randomly select 2 documents
]);
```

Aggregate With : $sort

Sort the documents based on the field in ascending or descending order

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },
  { $sort: { age: 1 } },                // 1 for ascending order
]);
```

Aggregate With : $sort

Sort the documents based on the field in descending order

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },
  { $sort: { age: -1 } },               // -1 for descending order
]);
```

Aggregate With : $sort

Sort the documents based on multiple fields, First by name in ascending order, then by age in ascending order

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } }, // Match students with age greater than 20
  { $sort: { name: 1, age: 1 } }, // Sort by name and then by age
]);
```

Aggregate With : $project

Reshape the documents by including, excluding, or adding new fields

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },
  { $sort: { name: 1, age: 1 } },
  { $project: { name: 1, class: 1, _id: 0 } }, // Project only name and class fields,
]);                                                    excluding _id
```

Aggregate With : $project

Reshape the documents by including, excluding, or adding new fields, and add custom field

```
db.students.aggregate([
  { $sort: { name: 1, age: 1 } },
  { $project: { name: 1, class: 1,  _id: 0,
      isValidAge: { $gt: ["$age", 20] },
    }, // Add a new field isValidAge that checks if age is greater than 20
  },
]);
```

Aggregate With : $skip + $limit

Skip a specified number of documents in the result set limiting the result set, useful in pagination

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },
  { $sort: { name: 1, age: 1 } },
  { $project: { name: 1, class: 1, _id: 0 } },
  { $skip: 4 }, // Skip the first 4 documents
  { $limit: 2 }, // Skip the first 4 documents and limit the result to 2 documents
]);
```

Aggregate With : $group

Group documents by a specified field and perform aggregations on the grouped data

```
db.students.aggregate([
  { $group: {
      _id: "$class",                    // Group by class field
    },
  },
]);
```

Group With : $sum

Group documents by a Class field and calculate the sum of a field in each group

```
db.students.aggregate([
  { $group: {
      _id: "$class", // Group by class field
      count: { $sum: 1 }, // Count custom field name
    },
  },
]);
```

Group With : $sum + $match

Group documents by a Class field and calculate the sum of a field in each group after matching a condition

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } }, // Match students with age greater than 20
  {
    $group: {
      _id: "$class", // Group by class field
      count: { $sum: 1 }, // Count custom field name
    },
  },
]);
```

Group With : $count

Group documents by a Class field and count the number of documents in each group

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",
      count: { $count: {} },
    }, // count is custom field name
  },
]);
```

Group With : $count + $sort

Group documents by a Class field and count the number of documents in each group, then sort by count

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",
      count: { $count: {} },
    }, // count is custom field name
  },
  { $sort: { count: 1 } }, // Sort by count in descending order
]);
```

Group With : $push

Group documents by a Class field and push all student names into an array

```
db.students.aggregate([
  { $group: {
      _id: "$class",
      students: { $push: "$name" }, // Push all student names into an array
    },
  },
]);
```

Group With : $push

Group documents by a Class field and push all student names into an array

```
db.students.aggregate([
  {    $group: {
      _id: "$class",
      students: { $push: "$$ROOT" }, // Push all student documents into an array using
    },                                                                          $$ROOT
  },
]);
```

Group With : $addToSet

Group documents by a Class field and add unique student names to an array

```
db.students.aggregate([
  {    $group: {
      _id: "$class",
      students: { $addToSet: "$name" }, // Add unique student names to an array
    },
  },
]);
```

Group With : $max

Group documents by a Class field and find the maximum age of students in each class

```
db.students.aggregate([
  { $group: {
      _id: "$class",
      Maximum_Student_Age: { $max: "$age" }, // Find the maximum age of students in each
    },                                                                          class
  },
]);
```

Group With : $min

Group documents by a Class field and find the minimum age of students in each class

```
db.students.aggregate([
  {    $group: {
      _id: "$class",
      Minimum_Student_Age: { $min: "$age" }, // Find the minimum age of students in each
    },                                                                          class
  },
]);
```

Group With : $avg

Group documents by a Class field and calculate the average age of students in each class

```
db.students.aggregate([
  { $group: {
      _id: "$class",                // Group by class field
      Average_Age: { $avg: "$age" }, // Calculate the average age of students in each
    },                                                                          class
  },
]);
```

Group With : $avg

Group documents without any specific field to get the average age of all students

```
db.students.aggregate([
  { $group: {
      _id: null,                    // No specific field to group by, so we use null
      Average_Age: { $avg: "$age" }, // Calculate the average age of all students
    },
  },
]);
```

Group With : $median

Calculate the median age of all students

```
db.students.aggregate([
  {
    $group: {
      _id: null,                     // No specific field to group by, so we use null
      Median_Age: {                  // Group by class field
        $median: {
          input: "$age",             // Field to calculate the median on
          method: "approximate",     // Use "exact" for exact median calculation
        },
      },
    },
  },
]);
```

Group With : $median

Group documents by a Class field and calculate the median age of students in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                  // Group by class field
      Median_Age: {                   // Custom field name for median age
        $median: {
          input: "$age",              // Field to calculate the median on
          method: "approximate",      // Use "exact" for exact median calculation
        },
      },
    },
  },
]);
```

Group With : $first

Group documents by a Class field and get the first student name in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                        // Group by class field
      First_Student: { $first: "$name" }, // Get the first student name in each class
    },
  },
]);
```

Group With : $last

Group documents by a Class field and get the last student name in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                        // Group by class field
      Last_Student: { $last: "$$ROOT" }, // Get the last student name in each class
    },
  },
]);
```

Group With : $top

Group documents by a Class field and get the top student based on age in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                    // Group by class field
      Top_students: {                   // Custom field name for top students
        $top: {
          output: ["$name", "$class", "$age"], // Fields to include in the output
          sortBy: { age: 1 },           // Sort by age in ascending order
        },
      },
    },
  },
]);
```

Group With : $topN

Group documents by a Class field and get the top N students based on age in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                    // Group by class field
      Top_students: {                   // Custom field name for top students
        $topN: {
          output: ["$name", "$class", "$age"], // Fields to include in the output
          sortBy: { age: 1 },           // Sort by age in ascending order
          n: 3,                         // Get the top 2 students in each class
        },
      },
    },
  },
]);
```

Group With : $bottom

Group documents by a Class field and get the bottom student based on age in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                    // Group by class field
      Top_students: {                   // Custom field name for bottom students
        $bottom: {
          output: ["$name", "$class", "$age"], // Fields to include in the output
          sortBy: { age: 1 },           // Sort by age in ascending order
        },
      },
    },
  },
]);
```

Group With : $bottomN

Group documents by a Class field and get the bottom N students based on age in each class

```
db.students.aggregate([
  {
    $group: {
      _id: "$class",                    // Group by class field
      Top_students: {                   // Custom field name for bottom students
        $bottomN: {
          output: ["$name", "$class", "$age"], // Fields to include in the output
          sortBy: { age: 1 },           // Sort by age in ascending order
          n: 3,                         // Get the bottom 3 students in each class
        },
      },
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name students & library for test Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", age: 23, class: "BCA" },
  { _id: 2, name: "Salman Khan", age: 24, class: "Btech" },
  { _id: 3, name: "Shahid Kapoor", age: 20, class: "BSc" },
  { _id: 4, name: "John Abraham", age: 19, class: "BCA" },
]);

db.library.insertMany([
  { _id: 1, book: "Atomic Habits", student_id: 1 },
  { _id: 2, book: "You Can", student_id: 2 },
  { _id: 3, book: "Do It Today", student_id: 4 },
]);
```

Join Example 1 : $lookup

Join Students Collection to Library Collection through Library Collection student_id Field to Students Collection _id Field

```
db.library.aggregate([
  {
    $lookup: {
      from: "students",                // Collection to join with Library Collection
      localField: "student_id",        // Field from Library Collection
      foreignField: "_id",             // Field from Students Collection
      as: "Student",                   // Name of the new array field to add to Library
    },                                                                    Collection
  },
]);
```

Join Example 2 : $lookup

Join Students Collection to Library Collection through Students Collection _id Field to Library Collection Student_id field

```
db.students.aggregate([
  {
    $lookup: {
      from: "library",                 // Collection to join with Students Collection
      localField: "_id",               // Field from Students Collection
      foreignField: "student_id",      // Field from Library Collection
      as: "Book",                      // Name of the new array field to add to
    },                                                      Students Collection
  },
]);
```

Join Example 3 : $lookup + $unwind

Join Student Collection to Library collection and use $unwind for if Book Array have only one document then remove Array

```
db.students.aggregate([
  {
    $lookup: {
      from: "library", // Collection to join with Students Collection
      localField: "_id", // Field from Students Collection
      foreignField: "student_id",// Field from Library Collection
      as: "Book",// Name of the new array field to add to Students Collection
    },
  },
  { $unwind: "$Book" }, // Unwind Book Array to remove array and make it a single
]);                                                                    document
```

Collection : For Testing Query

Create a Collection name customers & orders for test Operators

```
db.customers.insertMany([
  { _id: 101, name: "Akshay Kumar", city: "Delhi" },
  { _id: 102, name: "John Abraham", city: "Goa" },
  { _id: 103, name: "Salman Khan", city: "Mumbai" },
]);

db.orders.insertMany([
  { _id: 1, customer_id: 101, product: "Laptop", price: 35000 },
  { _id: 2, customer_id: 102, product: "Phone", price: 20000 },
  { _id: 3, customer_id: 101, product: "Phone ABC", price: 22000 },
  { _id: 4, customer_id: 103, product: "Tablet", price: 15000 },
  { _id: 5, customer_id: 102, product: "Tablet", price: 15000 },
]);
```

Join Example 4 : $lookup

Join Customer Collection to Oder Collection through Customer _id to order customer_id field

```
db.customers.aggregate([
  {
    $lookup: {
      from:  "orders",                  // Collection to join with Customers Collection
      localField: "_id",                // Field from Customers Collection
      foreignField: "customer_id",      // Field from Orders Collection
      as: "Orders",                     // Name of the new array field to add to
    },                                                  Customers Collection
  },
]);
```

Join Example 5 : $lookup

Join Library Collection to Students Collection through Library Collection student_id Field to Students Collection _id Field
and replace the root document with the merged object
This will merge the Student document into the Library document, removing the Student array

```
db.library.aggregate([
  {
    $lookup: {
      from: "students",             // Collection to join with Library Collection
      localField: "student_id",     // Field from Library Collection
      foreignField: "_id",          // Field from Students Collection
      as: "Student",                // Name of the new array field to add to Library
    },                                                                  Collection
  },
  {
    $replaceRoot: {                 // Replace the root document with a new object
      newRoot: {
        $mergeObjects: [{ $arrayElemAt: ["$Student", 0] }, "$$ROOT"],
                                    // Merge the first element of the Student array with
      },                                                        the current document
    },
  },
  { $project: { Student: 0 } },     // Exclude the Student array from the final output
]);
```

Collection : For Testing Query

Create a Collection name students for test Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", percentage: 52 },
  { _id: 2, name: "Salman Khan", percentage: 67 },
  { _id: 3, name: "Deepika Padukone", percentage: 83 },
  { _id: 4, name: "John Abraham", percentage: 49 },
  { _id: 5, name: "Katrina Kaif", percentage: 77 },
  { _id: 6, name: "Abhishek Bachan", percentage: 44 },
  { _id: 7, name: "Shahid Kaupr", percentage: 25 },
  { _id: 8, name: "Amir Khan", percentage: 38 },
  { _id: 9, name: "Kriti Sanon", percentage: 91 },
  { _id: 10, name: "Anushka Sharma", percentage: 59 },
]);
```

Example 1 : $bucket + output

Grouping students based on their percentage into different buckets

```
db.students.aggregate([
  {
    $bucket: {
      groupBy: "$percentage",              // Field to group by
      boundaries: [33, 45, 60, 80, 100],   // Boundaries for the buckets
      default: "Fail Students",      // Default bucket for values outside the boundaries
      output: {                            // Output fields for each bucket
        Count: { $sum: 1 },                // Count of students in each bucket
      },
    }, },]);
```

Example 2 : $bucket without output

Grouping students based on their percentage without specifying output fields

```
db.students.aggregate([
  {
    $bucket: {                       // Grouping students based on their percentage
      groupBy: "$percentage",        // Field to group by
      boundaries: [33, 45, 60, 80, 100], // Boundaries for the buckets
      default: "Fail Students", // Default bucket for values outside the boundaries
    },
  },]);
```

Example 3 : $bucketAuto

Automatically grouping students into a specified number of buckets based on their percentage

```
db.students.aggregate([
  {
    $bucketAuto: {                     // Automatically grouping students into buckets
      groupBy: "$percentage",          // Field to group by
      buckets: 3,                      // Number of buckets to create
      output: {
       Count: { $sum: 1 },             // Count of students in each bucket
       Average_percentage: { $avg: "$percentage" }, //Average % of students in each bucket
       Total_percentage: { $sum: "$percentage" }, // Total & of students in each bucket
      },
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name students for test Operators

```
db.students.insertMany([
  { _id: 1,firstname: "Akshay",  lastName: "Kumar", marks: [52, 59, 67], city: "Delhi"},
  { _id: 2,firstname: "Salman",  lastName: "Khan", marks: [77, 74, 69], city: "Mumbai"},
  { _id: 3,firstname: "Deepika", lastName: "Padukon",marks: [83, 64, 75],city: null},
  { _id: 4,firstname: "John",    lastName: "Abraham",marks: [49, 66, 58],city: "Goa"},
  { _id: 5, firstname: "Katrina",lastName: "Kaif", marks: [69, 74, 64] },
]);
```

Example 1 : $addfields + $concat

Fetch Document and concat field in contact first name & last name and how in fullName field

```
db.students.aggregate([
  {
    $addFields: {           // Add new field fullName
                           // Concatnate firstname and lastName with a space in between
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
    },
  },
]);
```

Example 2 : $addfields + $$REMOVE

Fetch Document and concat field in existing _id field, show full name in _id field,
Remove existing fields firstname and lastName after Concatnating them into fullName

```
db.students.aggregate([
  {
    $addFields: {                                    // Add new field fullName
      _id: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",                         // Remove firstname field
      lastName: "$$REMOVE",                          // Remove lastName field
    },
  },
]);
```

Example 3 : $addFieds + ifNull

Fetch Document and concat field in existing _id field, show full name in _id field, If city is null, remove it from the output

```
db.students.aggregate([
  {
    $addFields: {                                     // Add new field fullName
      _id: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",                          // Remove firstname field
      lastName: "$$REMOVE",                           // Remove lastName field
      city: { $ifNull: ["$city", "$$REMOVE"] }, // If city is null, remove it from the
    },                                                                          output
  },
]);
```

Example 4 : $addFieds + $cond (if else)

Fetch Document and concat field in existing _id field, show full name in _id field,
If city is "Delhi", remove it from the output, otherwise keep

```
db.students.aggregate([
  {
    $addFields: {
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",          // Remove firstname field
      lastName: "$$REMOVE",           // Remove lastName field
      city: {
        $cond: {                      // Conditional logic to check if city is "Delhi"
          if: {                       // Check if city is "Delhi"
            $eq: ["$city", "Delhi"],          },
          then: "$$REMOVE",           // If city is "Delhi", remove it from the output
          else: "$city",              // Otherwise, keep the city field
        },
      },
    },
  },
]);
```

Example 5 : $addFieds + $match

Match document with _id 1 and add fullName field, add firstname and lastName fields to fullName, and remove firstname and lastName fields

```
db.students.aggregate([
  { $match: { _id: 1 } },                                  // Match document with _id 1
  {
    $addFields: {
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",                               // Remove firstname field
      lastName: "$$REMOVE",                                // Remove lastName field
    },
  },
]);
```

Example 6 : $addFieds + custome field

Match document with _id 1 and add fullName field, and remove firstname and lastName fields Also, add a new field "age" to the "profile" object

```
db.students.aggregate([
  { $match: { _id: 1 } },                                 // Match document with _id 1
  {
    $addFields: {
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",                              // Remove firstname field
      lastName: "$$REMOVE",                               // Remove  lastName fields
      "profile.age": 30,                 // Add a new field "age" to the "profile" object
    },
  },
]);
```

Example 7 : $addFieds

add new value in marks array Match document with _id 1 and add fullName field, add a new value to the marks array, and remove firstname and lastName fields

```
db.students.aggregate([
  { $match: { _id: 1 } }, // Match document with _id 1
  {
    $addFields: {
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",
      lastName: "$$REMOVE",
      marks: { $concatArrays: ["$marks", [70]] }, // Add a new value 70 to the marks
    },                                                                    array
  },
]);
```

Example 8 : $addFieds

Match document with _id 1 and add fullName field, remove firstname and lastName fields, and calculate total marks

```
db.students.aggregate([
  { $match: { _id: 1 } },
  {
    $addFields: {
      fullName: { $concat: ["$firstname", " ", "$lastName"] },
      firstname: "$$REMOVE",
      lastName: "$$REMOVE",
      TatalMarks: { $sum: "$marks" }, // New Field TatalMarks in this fild show sum how
    },                                                                      marks
  },
]);
```

Example of : $unwind

Unwind the sizes array in the inventory collection
This will create a separate document for each size in the sizes array

```
// Create a collection named inventory with an array field sizes
db.inventory.insertMany([
  { _id: 1, item: "ABC", sizes: ["S", "M", "L"] },
  { _id: 2, item: "XYZ", sizes: ["S", "M", "L", "XL"] },
]);


db.inventory.aggregate([
  {
    $unwind: "$sizes", // Unwind the sizes array to create a separate document for each
  },                                                                        size
]);
```

Collection : For Testing Query

Create a Collection name students for test Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", age: 23, class: "BCA" },
  { _id: 2, name: "Salman Khan", age: 24, class: "Btech" },
  { _id: 3, name: "Shahid Kapoor", age: 20, class: "BSc" },
  { _id: 4, name: "John Abraham", age: 19, class: "BCA" },
  { _id: 5, name: "Amir Khan", age: 24, class: "Btech" },
  { _id: 6, name: "Salman Khan", age: 21, class: "BSc" },
  { _id: 7, name: "Suniel Shetty", age: 22, class: "BCA" },
  { _id: 8, name: "Kartik Aryan", age: 20, class: "Btech" },
]);
```

Example of : $out

This example filters students older than 20 and outputs them to a new collection called "valid_students".

```
db.students.aggregate([
  { $match: { age: { $gt: 20 } } },       // filter students older than 20
  { $out: "valid_students" },             // output to a new collection
]);
```

Example 2 : $out

This example outputs all students to a new collection called "valid_students".

```
// output all students to a new collection
db.students.aggregate([{ $out: "valid_students" }]);
```

Example 3 : $out

This example groups students by class and outputs the result to a new collection called "class_data".

```
db.students.aggregate([
  { $group: { _id: "$class", Student: { $push: "$name" } } }, // group by class
  { $out: "class_data" },                                     // output to a new collection
]);
```

Collection : For Testing Query

Create a Collection name personal, students1,students2 for test Operators

```
db.personal.insertMany([
  { _id: 1, city: "Delhi", phone: 3030320 },
  { _id: 2, city: "Gao", phone: 445522 },
  { _id: 3, city: "Mumbai", phone: 223347 },
  { _id: 4, city: "Delhi", phone: 303085 },
  { _id: 5, city: "Agra", phone: 221135 },
  { _id: 6, city: "Delhi", phone: 648877 },
  { _id: 7, city: "Goa", phone: 309988 },
  { _id: 8, city: "Mumbai", phone: 223355 },
]);


db.students1.insertMany([
  { _id: 1, name: "Kartik Aryan", role: "Student" },
  { _id: 2, name: "RajKumar Rao", role: "Student" },
]);

db.students2.insertMany([
  { _id: 1, name: "Salman Khan", role: "Alumnus" },
  { _id: 2, name: "Akshay Kumar", role: "Alumnus" },
]);
```

Example of : merge

This example merges students older than 20 into the "students" collection.

```
db.personal.aggregate([
  {
    $merge: {
      into: "students", // target collection
      on: "_id", // field to match on
      whenMatched: "merge", // options for matched ("merge", "replace", "keepExisting")
      whenNotMatched: "insert", // other options for unmatched ("insert", "discard")
    },
  },
]);
```

Example of : $unionWith

This example creates two collections: students1 and students2, and then uses $unionWith to combine them.

```
// This example combines documents from two collections: students1 and students2.
db.students1.aggregate([
  {
    $unionWith: {
      coll: "students2", // collection to union with
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name sales for test Operators

```
db.sales.insertMany([
  { _id: 1, product: "Mobile", price: 100, quantity: 10, region: "North" },
  { _id: 2, product: "Laptop", price: 200, quantity: 5, region: "South" },
  { _id: 3, product: "Mobile", price: 100, quantity: 15, region: "North" },
  { _id: 4, product: "Tablet", price: 50, quantity: 20, region: "East" },
  { _id: 5, product: "Desktop", price: 125, quantity: 10, region: "South" },
  { _id: 6, product: "Laptop", price: 200, quantity: 10, region: "West" },
]);
```

Example of : $facet

$facet is useful for aggregating data in different ways without running multiple queries
It allows you to perform multiple aggregation pipelines in parallel and return the results in a single document
Example: Get top 3 products by total sale, total revenue from all sales, and
total sales by region in a single query

```
db.sales.aggregate([
  {
    $facet: {
      topProducts: [ // get top 3 products by total sale
        {
          $group: {
            _id: "$product",
            totalSale: {
              $sum: { $multiply: ["$price", "$quantity"] },
            },
          },
        },

        { $sort: { totalSale: -1 } },
        { $limit: 3 },
      ],
      totalRevenue: [ // get total revenue from all sales
        {
          $group: {
            _id: null,
            totalRevenue: {
              $sum: { $multiply: ["$price", "$quantity"] },
            },
          },
        },
      ],
      salesByRegion: [ // get total sales by region
        {
          $group: {
            _id: "$region",
            count: { $sum: 1 },
          },
        },
        { $sort: { totalSale: -1 } },
      ],
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name students for test Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", class: "Btech", per: 52 },
  { _id: 2, name: "Salman Khan", class: "BCA", per: 67 },
  { _id: 3, name: "Deepika Padukone", class: "Btech", per: 83 },
  { _id: 4, name: "John Abraham", class: "Btech" },
  { _id: 5, name: "Katrina Kaif", class: "BCA", per: 77 },
  { _id: 6, name: "Abhishek Bachan", class: "BCA", per: 44 },
  { _id: 7, name: "Shahid Kapoor", class: "Btech" },
  { _id: 8, name: "Amir Khan", class: "Btech", per: 38 },
]);
```

Example of : $fill

It can fill missing values in a field based on different methods like linear interpolation, last observation carried forward (locf), or a constant value It is useful when you have missing data in your dataset and you want to fill it with

```
db.students.aggregate([
  {
    $fill: {
      output: {
        per: { value: 25 }, // Fill missing 'per' values with 25
      },
    },
  },
]);
```

Example of : $fill with locf

Fill missing 'per' values with the last observation carried forward (locf) This method fills missing values with the last non-null value encountered in the sequence It is useful when you want to carry forward the last known value in a time series or sequential data For example, if the last known percentage was 52, it will fill the next missing

```
db.students.aggregate([
  {
    $fill: {
      output: {
        per: { method: "locf" }, // Fill missing 'per' values with 25
      },
    },
  },
]);
```

Example of : $fill with linear

This method fills missing values by interpolating between the last known value and the next known value It is useful when you want to fill missing values in a continuous data series // For example, if the last known percentage was 52 and the next known percentage is 83, it will fill the missing value with a linear interpolation between these two values

```
db.students.aggregate([
  {
    $fill: {
      sortBy: { _id: 1 }, // Ensure the documents are sorted by _id before filling
      partitionBy: "$class", // Fill missing 'per' values within each class
      output: {
        per: { method: "linear" }, // Fill missing 'per' values with 25
      },
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name sales for test Operators

```
db.sales.insertMany([
  { _id: 1, product: "Mobile", price: 100, quantity: 52 },
  { _id: 2, product: "Laptop", price: 150, quantity: 65 },
  { _id: 3, product: "Tablet", price: 120, quantity: 30 },
]);
```

Addition Operator : $add

Adding two fields or value together (performing addition operation)

```
db.sales.aggregate([
  {
    $project: {
      product: 1,                                  // Including product field in the output
      price: 1,                                    // Including price field in the output
      quantity: 1,                                 // Including quantity field in the output
      Result: { $add: ["$price", "$quantity"] }, // Adding price and quantity fields
    },
  },
]);
```

Subtraction Operator : $substract

Subtracting one field from another (performing subtraction operation)

```
db.sales.aggregate([
  {
    $project: {
      Result: { $subtract: ["$price", "$quantity"] }, // Subtracting quantity from price
    },
  },
]);
```

Multiplication Operator : $multiply

Multiplying two fields together (performing multiplication operation)

```
db.sales.aggregate([
  {
    $project: {
      Result: { $multiply: ["$price", "$quantity"] }, // Multiplying price and quantity
    },                                                                    fields
  },
]);
```

Division Operator : $divide

Dividing one field by another (performing division operation)
Note: Ensure that the divisor is not zero to avoid division by zero errors.

```
db.sales.aggregate([
  {
    $project: {
      Result: { $divide: [10, 2] }, // Dividing 10 by 2
    },
  },
]);
```

Division remainder Operator : $mod

Finding the remainder of a division operation (modulus operation)
Note: Ensure that the divisor is not zero to avoid division by zero errors.

```
db.sales.aggregate([
  {
    $project: {
      Result: { $mod: [10, 3] }, // Finding the remainder of 10 divided by 3
    },
  },
]);
```

Power Operator : $pow

Raising a number to the power of another number (exponentiation operation)
Note: The first argument is the base, and the second argument is the exponent.

```
db.sales.aggregate([
  {
    $project: {
      Result: { $pow: [2, 3] }, // Raising 2 to the power of 3
    },
  },
]);
```

Square Root Operator : $sqrt

Calculating the square root of a number Note: The argument should be a non-negative number.

```
db.sales.aggregate([
  {
    $project: {
      Result: { $sqrt: 16 }, // Calculating the square root of 16
    },
  },
]);
```

Round up Operator : $ceil

Rounding a number up to the nearest integer

```
db.sales.aggregate([
  {
    $project: {
      Result: { $ceil: 9.2 }, // Rounding 9.2 up to the nearest integer
    },
  },
]);
```

Round down Operator : $floor

Rounding a number down to the nearest integer

```
db.sales.aggregate([
  {
    $project: {
      Result: { $floor: 9.2 }, // Rounding 9.2 down to the nearest integer
    },
  },
]);
```

Round Operator : $round

Rounding a number to the nearest integer or specified decimal places if point value less thane 0.5 then it will round down otherwise it will round up

```
db.sales.aggregate([
  {
    $project: {
      Result: { $round: 9.2 }, // Rounding 9.2 to the nearest integer
    },
  },
]);
```

Truncating : $trunc

Truncating a number to remove the decimal part

```
db.sales.aggregate([
  {
    $project: {
      Result: { $trunc: 9.2 }, // Truncating 9.2 to remove the decimal part
    },
  },
]);
```

Collection : For Testing Query

Create 3 Collection name students, students2, students3 for testing String Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", dob: "jan 10 2010" },
  { _id: 2, name: "Salman Khan", dob: "2010-02-03" },
  { _id: 3, name: "Deepika Padukone", dob: "june 15 2010" },
  { _id: 4, name: "John Abraham", dob: "WED jan 31 10:05:28 +03:30 2010" },
  { _id: 5, name: "Katrina Kaif", dob: "dec 22 2010" },
  { _id: 6, name: "    Test    " },
]);

db.students2.insertMany([
  { _id: 1, firstName: "Akshay", lastName: "Kumar", age: 25 },
  { _id: 2, firstName: "Salman", lastName: "Khan", age: 23 },
  { _id: 3, firstName: "Deepika", lastName: "Padukone", age: 24 },
  { _id: 4, firstName: "John", lastName: "Abraham", age: 25 },
  { _id: 5, firstName: "Katrina", lastName: "Kaif", age: 23 },
]);

db.students3.insertMany([
  { _id: 1, name: "Akshay Kumar", dob: ISODate("2008-01-15T08:15:39.736Z")},
  { _id: 2, name: "Salman Khan", dob: ISODate("2009-08-01T08:15:39.736Z")},
]);
```

Upper Case Operator : $toUpper

Converts a string to uppercase.

```
db.students.aggregate([
  {
    $project: {
      upperCaseName: { $toUpper: "$name" },
    },
  },
]);
```

Lower Case Operator : $toLower

Converts a string to lowercase.

```
db.students.aggregate([
  {
    $project: {
      lowerCaseName: { $toLower: "$name" },
    },
  },
]);
```

String Length Operator : $strLenBytes

Returns the length of a string in bytes.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      lenghtInBytes: { $strLenBytes: "$name" },
    },
  },
]);
```

String Length in Code Point Operator : $strLenCP

Returns the length of a string in code points. This is useful for strings with multi-byte characters.
It counts each character as one, regardless of how many bytes it uses.
For example, "A" is 1 code point and 1 byte, while "𐌾" (a character from the Gothic script) is 1 code point but 4 bytes.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      lenghtInCodePoints: { $strLenCP: "$name" },
    },
  },
]);
```

Compare two string Operator : $strcasecmp

Compares two strings in a case-insensitive manner. Returns 0 if the strings are equal, a negative number if the first string is less than the second, and a positive number if the first string is greater than the second.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      comparison: { $strcasecmp: ["$name", "Akshay Kumar"] }, // Case-insensitive
comparison
    },
  },
]);
```

Sub String Operator : $substrBytes

Extracts a substring from a string based on byte position.

```
db.students.aggregate([
  {
    $project: {
      substring: { $substrBytes: ["$name", 0, 5] }, // Extracting first 5 bytes
    },
  },
]);
```

Sub String Operator : $substrCP

Replaces the first occurrence of a specified string.

```
db.students.aggregate([
  {
    $project: {
      substring: { $substrCP: ["$name", 0, 5] }, // Extracting first 5 code points
    },
  },
]);
```

Replace String Operator : $replaceOne

Replaces the first occurrence of a specified string.

```
db.students.aggregate([
  {
    $project: {
      updateString: {
        $replaceOne: { input: "$name", find: "Khan", replacement: "Kapoor," }, //
Replacing first occurrence of "Khan" with "Kapoor,"
      },
    },
  },
]);
```

Replace String Operator : $replaceAll

Replaces all occurrences of a substring within a string.

```
db.students.aggregate([
  {
    $project: {
      updateString: {
        $replaceAll: {
          input: "My Name is Khan his name is Khan", // in real life we use field like
          find: "Khan",                                    "$name" not a string
          replacement: "Kapoor,",
        },
      },
    },
  },
]);
```

Split String Operator : $split

Splits a string into an array of substrings based on a specified delimiter.

```
db.students.aggregate([
  {
    $project: {
      words: { $split: ["$name", " "] }, // Splitting by space
    },
  },
]);
```

Concat String Operator : $concat

Concatnates two or more strings together.

```
db.students2.aggregate([
  {
    $project: {
      fullName: { $concat: ["$firstName", " ", "$lastName"] },
    },
  },
]);
```

Value to String Operator : $toString

Converts a value to a string. This is useful for converting non-string values to strings.

```
db.students2.aggregate([
  {
    $project: {
      stringField: { $toString: "$age" },
    },
  },
]);
```

Left Trim Operator : $ltrim

Removes whitespace or specified characters from the beginning of a string. left trim

```
db.students.aggregate([
  {
    $project: {
      trinmmed: { $ltrim: { input: "$name", chars: " " } }, // Trimming leading spaces
    },
  },
]);
```

Right Trim Operator : $rtrim

Removes whitespace or specified characters from the end of a string. right trim

```
db.students.aggregate([
  {
    $project: {
      trinmmed: { $rtrim: { input: "$name", chars: " " } },
    },
  },
]);
```

Trim Operator : $trim

Removes whitespace or specified characters from both ends of a string.

```
db.students.aggregate([
  {
    $project: {
      trimmed: { $trim: { input: "$name", chars: " " } },
    },
  },
]);
```

String to Date Operator : $dateFromString

Converts a string to a date object. This is useful for parsing date strings into MongoDB's date format.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      date: { $dateFromString: { dateString: "$dob" } },
    },
  },
]);
```

Date to String Operator : $dateToString

Converts a date object to a string in a specified format. Formatting date to "YYYY-MM-DD"

```
db.students3.aggregate([
  {
    $project: {
      name: 1,
      date: { $dateToString: { format: "%Y-%m-%d", date: "$dob" } },
    },
  },
]);
```

Find Index of String Operator : $indexOfBytes

Finds the index of a substring within a string, counting bytes.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      index: { $indexOfBytes: ["$name", "K"] },
    },
  },
]);
```

Find Index of String with start and end Operator : $indexOfByetes

Finds the index of a substring within a string, counting bytes.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      index: { $indexOfBytes: ["$name", "K"] },
    },
  },
]);
```

Find Index of String with start and end Operator : $indexOfByetes

Finds the index of a substring within a string, counting bytes, starting from a specific position and ending at another.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      index: { $indexOfBytes: ["$name", "k", 6, 15] },
    },
  },
]);
```

Find Index of String with Code Point Operator : $indexOfCP

Finds the index of a substring within a string, counting code points.

```
db.students.aggregate([ {
    $project: {
      name: 1,
      index: { $indexOfCP: ["$name", "man"] },
    },
  },
]);
```

Regular Expressions Operator : $regexMatch

Matches a string against a regular expression and returns the first match.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      isMatch: { $regexMatch: { input: "$name", regex: "^Kat" } },
    },
  },
]);
```

Regular Expressions Operator : $regexFind

Finds the first occurrence of a substring that matches a regular expression.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      isMatch: { $regexFind: { input: "$name", regex: "^Sa" } },
    },
  },
]);
```

Regular Expressions Operator : $regexFindAll

Finds all occurrences of substrings that match a regular expression.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      isMatch: { $regexFindAll: { input: "$name", regex: "K" } },
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name students for testing Date Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", dob: ISODate("2008-01-15T08:15:39.736Z")},
  { _id: 2, name: "Salman Khan", dob: ISODate("2009-08-01T08:15:39.736Z")},
]);
```

Date : Format Specifiers

The following format specifiers are available for use in the <formatString>:

Specifiers	Description	Possible Values
%b	Abbreviated month name (3 letters) *New in version 7.0*.	jan-dec
%B	Full month name *New in version 7.0*.	january-december
%d	Day of month (2 digits, zero padded)	01-31
%G	Year in ISO 8601 format	0000-9999
%H	Hour (2 digits, zero padded, 24-hour clock)	00-23
%j	Day of year (3 digits, zero padded)	001-366
%L	Millisecond (3 digits, zero padded)	000-999
%m	Month (2 digits, zero padded)	01-12
%M	Minute (2 digits, zero padded)	00-59
%S	Second (2 digits, zero padded)	00-60
%u	Day of week number in ISO 8601 format (1-Monday, 7-Sunday)	1-7
%U	Week of year (2 digits, zero padded)	00-53
%V	Week of Year in ISO 8601 format	01-53
%w	Day of week (1-Sunday, 7-Saturday)	1-7
%Y	Year (4 digits, zero padded)	0000-9999
%z	The timezone offset from UTC.	+/-[hh][mm]
%Z	The minutes offset from UTC as a number. For example, if the timezone offset (+/-[hhmm]) was +0445, the minutes offset is +285.	+/-mmm
%%	Percent Character as a Literal	%

Date Operator : $year, $month, $week

these operators extract the year, month, and week from a date field.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      Year: { $year: "$dob" },
      Month: { $month: "$dob" },
      Week: { $week: "$dob" },
    },
  },
]);
```

Date Operator : $dayOfMonth, $dayOfWeek, $dayOfYear

these operators extract the day of the month, day of the week, and day of the year from a date field.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      day: { $dayOfMonth: "$dob" },
      dayofweek: { $dayOfWeek: "$dob" },
      dayofyear: { $dayOfYear: "$dob" },
    },
  },
]);
```

Time Operator : $hour, $minute, $second, $millisecond

these operators extract the hour, minute, second, and millisecond from a date field.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      Hour: { $hour: "$dob" },
      Minutes: { $minute: "$dob" },
      Second: { $second: "$dob" },
      MilliSecond: { $millisecond: "$dob" },
    },
  },
]);
```

Date Add Operator : $dateAdd

This operator add days, months, years, etc. to a date field.

```
db.students.aggregate([ {
    $project: {
      name: 1,
      newDate: {
        $dateAdd: {
          startDate: "$dob",
          unit: "day", // year,quarter,week,month,day,hour,minite,second,millisecond
          amount: 5,
        },
      },
    },
  },
]);
```

Date Subtract Operator : $dateSubtract

This operator subtracts days, months, years, etc. from a date field.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      newDate: {
        $dateSubtract: {
          startDate: "$dob",
          unit: "month", // year,quarter,week,month,day,hour,minite,second,millisecond
          amount: 1,
        },
      },
    },
  },
]);
```

Date Difference Operator : $dateDiff

this operator calculates the difference between two dates in a specified unit.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      difference: {
        $dateDiff: {
          startDate: "$dob",
          endDate: ISODate("2025-12-31T00:00:00Z"),
          unit: "year", // year,quarter,week,month,day,hour,minite,second,millisecond
        },
      },
    },
  },
]);
```

Date Create Operator : $dateFromParts

this operator constructs a date from individual components such as year, month, day, hour, minute, second, millisecond, and timezone. It can be used to create a date from separate fields or to manipulate date components.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      ConstructeDate: {
        $dateFromParts: {
          year: 2025,
          month: 1,
          day: 7,
          hour: 15,
          minute: 45,
          second: 40,
          millisecond: 150,
          timezone: "Europe/London", //UTC
        },
      },
    },
  },
]);
```

Date To Parts Operator : $dateToParts

this operator breaks down a date into its individual components such as year, month, day, hour, minute, second, millisecond, and timezone.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      ConstructeDate: {
        $dateToParts: {
          date: "$dob",
          timezone: "Europe/London", // by default UTC
        },
      },
    },
  },
]);
```

Date To Parts Operator : $dateTrunc

this operator truncates a date to a specified unit, such as year, quarter, week, month, day, hour, minute, second, or millisecond.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      truncatedDate: {
        $dateTrunc: {
          date: "$dob",
          unit: "month", // year,quarter,week,month,day,hour,minite,second,millisecond
        },
      },
    },
  },
]);
```

Date To String Operator : $dateToString

this operator formats a date as a string according to a specified format.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      Date: {
        $dateToString: {
          date: "$dob",
          format: "%d-%m-%Y", // Date Specific Format
        },
      },
    },
  },
]);
```

String to Date Operator : $toDate

this operator converts a string or number to a date object. It can be used to convert

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      Date: {
        $toDate: "2018-01-20",
      },
    },
  },
]);
```

Date Operator : $isoDayOfWeek

this operator extracts the ISO day of the week from a date field, where Monday is

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      isoDayOfWeek: {
        $isoDayOfWeek: "$dob",
      },
    },
  },
]);
```

Date Operator : $isoWeek

this operator extracts the ISO week number from a date field, where the first week of the year is the week containing the first Thursday.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      isoWeekYear: {
        $isoWeekYear: "$dob",
      },
    },
  },
]);
```

Collection : For Testing Query

Create 3 Collection name students, students2, students3 for testing Array Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", hobbies: ["music", "travel", "painting"] },
  { _id: 2, name: "Salman Khan", hobbies: ["music", "travel", "books", "football"]},
]);

db.students2.insertMany([
  { _id: 1, name: "Akshay Kumar", marks: [55, 62, 52, 81] },
  { _id: 2, name: "Salman Khan", marks: [68, 88, 54, 51] },
]);

db.students3.insertMany([
  { _id: 1, name: "Akshay Kumar",
    subjects: ["Math", "Science"],
    extraSubjects: ["History", "Geography"],
  },
  { _id: 2, name: "Salman Khan",
    subjects: ["Math", "Science"],
    extraSubjects: ["Music", "Civics"],
  },
]);

db.students4.insertMany([
  {
    _id: 1, name: "Akshay Kumar",
    subjects: ["Math", "Science", "History"],
    marks: [85, 77, 82],
  },
  {
    _id: 2, name: "Salman Khan",
    subjects: ["Math", "Science", "Music"],
    marks: [88, 81, 95],
  },
]);

db.students5.insertMany([
  {
    _id: 1,  name: "Akshay Kumar",
    subjects: [
      ["Math", 85],
      ["Science", 77],
      ["History", 82],
    ],
  },
  {
    _id: 2,  name: "Salman Khan",
    subjects: [
      ["Math", 88],
      ["Science", 81],
      ["Music", 95],
    ],
  },
]);

db.students6.insertMany([
  { _id: 1, studentInfo: { name: "Akshay Kumar", age: 25 } },
  { _id: 2, studentInfo: { name: "Salman Khan", age: 26 } },
]);
```

Array Element At Operator : $arrayElemAt

This operator is used to access an element at a specific index in an array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      selectedHobby: {
        $arrayElemAt: ["$hobbies", 2], // Retrieves the hobby at index 2
      },
    },
  },
]);
```

Array First + N Operator : $firstN

This operator retrieves the first N elements from an array. example: if pass n as 2 then it will return first 2 elements of the array

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      topdHobby: {
        $firstN: { input: "$hobbies", n: 2 }, // Retrieves the first 2 hobbies
      },
    },
  },
]);
```

Array Last + N Operator : $lastN

This operator retrieves the last N elements from an array. example: if pass n as 2 then it will return last 2 elements of the array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      lastHobby: {
        $lastN: { input: "$hobbies", n: 2 }, // Retrieves the last 2 hobbies
      },
    },
  },
]);
```

Array Last + N Operator : $lastN

This operator retrieves the last N elements from an array. example: if pass n as 2 then it will return last 2 elements of the array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      lastHobby: {
        $lastN: { input: "$hobbies", n: 2 }, // Retrieves the last 2 hobbies
      },
    },
  },
]);
```

Maximum + N Operator : $maxN

This operator retrieves the N largest elements from an array. example: if pass n as 2 then it will return top 2 elements of the array.

```
db.students2.aggregate([
  {
    $project: {
      name: 1,
      topMarks: {
        $maxN: { input: "$marks", n: 2 }, // Retrieves the top 2 marks
      },
    },
  },
]);
```

Minimum + N Operator : $minN

This operator retrieves the N smallest elements from an array. example: if pass n as 2 then it will return bottom 2 elements From Arr.

```
db.students2.aggregate([
  {
    $project: {
      name: 1,
      topMarks: {
        $minN: { input: "$marks", n: 2 }, // Retrieves the bottom 2 marks
      },
    },
  },
]);
```

Array Slice Operator : $slice

This operator is used to retrieve a subset of an array. It can be used to get a specific number of elements from the start or end of the array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      selectedHobby: {
        $slice: ["$hobbies", 1, 2], // Retrieves 2 hobbies starting from index 1
      },
    },
  },
]);
```

Slice Operator with negative index : $slice

This allows you to retrieve elements from the end of the array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      selectedHobby: {
        $slice: ["$hobbies", -2, 1], // Retrieves 2 hobbies starting from the last
      },                                                                  element
    },
  },
]);
```

Sort Operators With String Array : $sortArray

This operator sorts the elements of an array based on a specified order.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      shortedHobbies: {
        $sortArray: { input: "$hobbies", sortBy: 1 }, // Sorts hobbies in ascending
      },                                                order, for descending order use -1
    },
  },
]);
```

Sort Operators With Number Array : $sortArray

This operator sorts the elements of an array based on a specified order.

```
db.students2.aggregate([
  {
    $project: {
      name: 1,
      shortedHobbies: {
        $sortArray: { input: "$marks", sortBy: 1 }, // Sorts marks in ascending order,
      },                                                  for descending order use -1
    },
  },
]);
```

Array Reverse Operators : $reverseArray

This operator reverses the order of elements in an array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      reversedHobbies: {
        $reverseArray: "$hobbies", // Reverses the order of hobbies
      },
    },
  },
]);
```

Array Size Operators : $size

This operator returns the number of elements in an array.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      HobbiesCount: {
        $size: "$hobbies", // Returns the count of hobbies in the array
      },
    },
  },
]);
```

Array In Operator : $in

This operator checks if a specified value exists in an array and returns true or false.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      check: {
        $in: ["football", "$hobbies"], // Checks if "football" is in the hobbies array
      },
    },
  },
]);
```

Array Search Operator : $indexOfArray

This operator returns the index of the first occurrence of a specified value in an array. If the value is not found, it returns -1.

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      index: {
        $indexOfArray: ["$hobbies", "football"], // Returns the index of "football" in
      },                                                the hobbies array
    },
  },
]);
```

Index Of Array With Range : $indexOfArray

This operator allows you to specify a starting index and a maximum number of elements to search.
Returns the index of "football" in the hobbies array, starting from index 1 and searching up to index 4

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      index: {
        $indexOfArray: ["$hobbies", "football", 1, 4], // Searches for "football"
      },                                                   starting from index 1 and up to 4 elements
    },
  },
]);
```

Is Array Operator : $isArray

This operator checks if a specified field is an array and returns true or false.

```
db.students.aggregate([
  {
    $project: { isArray: { $isArray: "$hobbies" } }, // Checks if "hobbies" is an array
  },
]);
```

Map (Loop) Operator : $map

This operator applies a specified expression to each element in an array and returns a new array with the results.
Example: Convert all hobbies to uppercase

```
db.students.aggregate([
  {
    $project: {
      upperCaseHobbies: {
        $map: {
          input: "$hobbies", // Input array of hobbies
          as: "hobbies", // Variable name for each element in the input array
          in: { $toUpper: "$$hobbies" }, // Converts each hobby to uppercase
        },
      },
    },
  },
]);
```

Map (Loop) Operator With Addition : $map

This operator applies a specified expression to each element in an array and returns a new array with the results.
Example: Add 2 to each mark in the marks array

```
db.students2.aggregate([
  {
    $project: {
      newMarks: {
        $map: {
          input: "$marks",
          as: "marks",
          in: { $add: ["$$marks", 2] }, // Adds 2 to each mark in the marks array
        },
      },
    },
  },
]);
```

Filter Operator : $filter

This operator filters the elements of an array based on a specified condition and returns a new array with the elements that match the condition.

```
db.students2.aggregate([
  {
    $project: {
      AboveMarks: {
        $filter: {
          input: "$marks",
          as: "marks",
          cond: { $gte: ["$$marks", 60] }, // Filters marks greater than or equal to 60
          limit: 1, // Optional: limits the number of elements in the result
        },
      },
    },
  },
]);
```

Reduce Operator : $reduce

This operator applies an expression to each element in an array, accumulating a single result.
Example: Calculate the total marks by summing up all elements in the marks array

```
db.students2.aggregate([
  {
    $project: {
      TotalMarks: {
        $reduce: {
          input: "$marks",
          initialValue: 0,
          in: { $add: ["$$value", "$$this"] }, // Sums up all marks in the marks array
        },
      },
    },
  },
]);
```

Reduce Operator with string contact nation : $reduce

This operator applies an expression to each element in an array, accumulating a single result.
Example: Concatnate all hobbies into a single string

```
db.students.aggregate([
  {
    $project: {
      CombinedHobbies: {
        $reduce: {
          input: "$hobbies",
          initialValue: "",
          in: { $concat: ["$$value", "$$this"] }, // Concatnates all hobbies into a
        },                                                                single string
      },
    },
  },
]);
```

Range Operator : $range

This operator generates an array of numbers within a specified range.

```
db.students.aggregate([
  {
    $project: {
      numbers: {
        $range: [1, 6, 2], // Generates an array of numbers from 1 to 6 with a step of 2
      },
    },
  },
]);
```

Range Operator with negative step : $range

This operator generates an array of numbers within a specified range, allowing for negative steps.

```
db.students.aggregate([{
    $project: {
      numbers: {
        $range: [10, 0, -2], // Generates an array of numbers from 10 to 0 with a step
      },                                                                          of -2
    },
  },]);
```

Contact Operator : $concatArrays

This operator Concatnates two or more arrays into a single array.

```
db.students3.aggregate([
  {
    $project: {
      name: 1,
      allSubjects: {
        $concatArrays: ["$subjects", "$extraSubjects"], // Combines subjects and
      },                                                  extraSubjects into a single array
    },
  },
]);
```

Zip Operator : $zip

This operator combines multiple arrays into an array of arrays, where each inner array contains elements from the input arrays at the same index.

```
db.students4.aggregate([
  {
    $project: {
      name: 1,
      data: {
        $zip: { inputs: ["$subjects", "$marks"] }, // Combines subjects and marks into
      },                                                                an array of arrays
    },
  },
]);
```

Array to Object Operator : $arrayToObject

This operator converts an array of key-value pairs into an object. Converts the subjects array into an object where each subject is a key and its corresponding mark is the value

```
db.students5.aggregate([
  {
    $project: {
      name: 1,
      SubjectInfo: {
        $arrayToObject: "$subjects",
      },
    },
  },
]);
```

Object to Array Operator : $objectToArray

This operator converts an object into an array of key-value pairs.

```
db.students6.aggregate([
  {
    $project: {
      name: 1,
      SubjectInfo: {
        $objectToArray: "$studentInfo", // Converts the studentInfo object into an array
      },                                                                of key-value pairs
    },
  },
]);
```

Collection : For Testing Query

Create a Collection name students for testing Type Operators

```
db.students.insertMany([
  {
    _id: 1,
    name: "Akshay Kumar",
    age: "22",
    marks: 82,
    pass: true,
    dob: ISODate("2002-03-27T16:58:51.538Z"),
  },
  {
    _id: 2,
    name: "Salman Khan",
    age: "23",
    marks: 33.58,
    pass: false,
    dob: ISODate("2001-05-15T18:15:35.264Z"),
  },
]);
```

To String Operator : $toString

converts a value to a string. Example: Convert the 'name' field to a string

```
db.students.aggregate([
  {
    $project: {
      StringValue: {
        $toString: "$name", // Convert 'name' to string
      },
    },
  },
]);
```

To Integer Operator : $toInt

converts a value to an integer. Example: Convert the 'age' field to an integer

```
db.students.aggregate([
  {
    $project: {
      IntValue: {
        $toInt: "$age", // Convert 'age' to integer
      },
    },
  },
]);
```

To Integer Operator with decimal values : $toInt

converts a decimal value to an integer. Example: Convert the 'marks' field to an integer

```
db.students.aggregate([  {
    $project: {
      IntValue: {
        $toInt: "$age", // Convert 'age' to integer
      },
    },
  },
]);
```

To Integer Operator With Bool field : $toInt

converts a boolean value to an integer. Example: Convert the 'pass' field to an integer

```
db.students.aggregate([{
    $project: {
      IntValue: {
        $toInt: "$pass", // Convert 'pass' to integer (true -> 1, false -> 0)
      },
    },
  },
]);
```

To Long Integer Operator : $toLong

converts a value to a long integer. Example: Convert the 'marks' field to a long integer

```
db.students.aggregate([{
    $project: {
      IntValue: {
        $toLong: "$marks", // Convert 'marks' to long integer
      },
    },
  },
]);
```

To Double Operator : $toDouble

converts a value to a double. Example: Convert the 'marks' field to a double

```
db.students.aggregate([{
    $project: {
      doubleValue: {
        $toDouble: "$marks", // Convert 'marks' to double
      },
    },
  },
]);
```

To Decimal Operator : $toDecimal

converts a value to a decimal. Example: Convert the 'marks' field to a decimal

```
db.students.aggregate([{
    $project: {
      decimalValue: {
        $toDecimal: "$marks", // Convert 'marks' to decimal
      },
    },
  },
]);
```

Type Operators : $type

returns the BSON data type of a field. It can be used in aggregation pipelines to identify the type of a field

```
db.students.aggregate([  {
    $project: {
      fildType: {
        $type: "$name", // Get the type of 'name' field
      },
    },
  },]);
```

To Bool Operators : $toBool

converts a value to a boolean. Example: Convert the 'pass' field to Boolean

```
db.students.aggregate([
  {
    $project: {
      boolValue: {
        $toBool: "$pass", // Convert 'pass' to boolean
      },
    },
  },
]);
```

To Object ID Operators : $toObjectId

converts a value to an ObjectId. It can be used in aggregation pipelines to transform data types Example: Convert a string to ObjectId Note: Ensure the string is a valid ObjectId format before conversion

```
db.students.aggregate([
  {
    $project: {
      ObjectId: {
        $toObjectId: "6889bd7e7a007f86246c4bd0", // Convert string to ObjectId
      },
    },
  },
]);
```

Convert Operators : $convert

provides more flexibility in type conversion. It allows specifying the input, target type, and options

```
db.students.aggregate([
  {
    $project: {
      ConvertedValue: {
        $convert: {
          input: "$age", // Input value to convert
          to: "int", // Target type
        },
      },
    },
  },
]);
```

Is Number Operators : $isNumber

checks if a value is a number. It can be used in aggregation pipelines to filter or project numeric values
Example: Check if the 'dob' field is a number (it should return false since

```
db.students.aggregate([
  {
    $project: {
      isNumber: {
        $isNumber: "$dob", // Check if 'dob' is a number
      },
    },
  },
]);
```

Collection : For Testing Query

Create 3 Collection name products, users, students for testing Logic Operators

```
db.products.insertMany([
  { _id: 1, name: "Laptop", price: 1200, discounted: true },
  { _id: 2, name: "Phone", price: 800, discounted: false },
  { _id: 3, name: "Tablet", price: 600, discounted: true },
]);

db.users.insertMany([
  { _id: 1, name: "Akshay Kumar", email: "akshay@email.com" },
  { _id: 2, name: "Salman Khan" },
  { _id: 3, name: "John Abraham", email: null },
]);

db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", percentage: 85 },
  { _id: 2, name: "Salman Khan", percentage: 72 },
  { _id: 3, name: "John Abraham", percentage: 58 },
  { _id: 4, name: "Shahid Kapoor", percentage: 30 },
]);
```

$cond Operator With : if-then-else

Thise operator allows you to perform conditional Logic in aggregation pipelines. Example: Categorize products based on price

```
db.products.aggregate([
  {
    $project: {
      name: 1,
      price: 1,
      priceCategory: {
        $cond: {
          if: { $gt: ["$price", 1000] }, // Check if price is greater than 1000
          then: "Expensive", // If true, categorize as expensive
          else: "Affordable", // If not, categorize as affordable
        },
      },
    },
  },
]);
```

Second Example of : if-then-else

This operator allows you to apply conditional logic in aggregation pipelines. Example: Apply a discount if the product is discounted

```
db.products.aggregate([{
    $project: {
      name: 1,
      price: 1,
      OriginalPrice: "$price", // Keep original price
      finalPrice: {
        $cond: {
          if: "$discounted", // Check if the product is discounted
          then: { $multiply: ["$price", 0.9] }, // Apply 10% discount
          else: "$price", // No discount
        },
      },
    },
  },
]);
```

If Null Operators : $ifNull

This operator allows you to provide a default value if a field is null. Example: Provide a default value for email if it is null

```
db.users.aggregate([
  {
    $project: {
      name: 1,
      Email: { $ifNull: ["$email", "No Email Provided"] }, // Use $ifNull to provide a
    },                                                      default value if email is null
  },
]);
```

Switch Operators : $switch

This operator allows you to perform multiple conditional checks. Example: Assign grades based on percentage
Here, we categorize students based on their percentage scores into different grades.

```
db.students.aggregate([
  {
    $project: {
      name: 1,       // Include the student's name
      percentage: 1, // Include the student's percentage
      grade: {       // Use $switch to categorize students based on their percentage
        $switch: {
          branches: [ // Define the conditions for each grade
            { case: { $gte: ["$percentage", 80] }, then: "Merit" },
            { case: { $gte: ["$percentage", 60] }, then: "Ist Devision"},
            { case: { $gte: ["$percentage", 45] }, then: "IInd Devision" },
            { case: { $gte: ["$percentage", 33] }, then: "IIIrd Devision" },
          ],
          default: "Fail", // Default case if none of the conditions match
        },
      },
    },
  },
]);



// If percentage is 80 or above, assign "Merit"
// If percentage is 60 or above, assign "Ist Devision"
// If percentage is 45 or above, assign "IInd Devision"
// If percentage is 33 or above, assign "IIIrd Devision"
```

Capped Collection : capped

This collection will store log entries with a maximum size of 50KB and a maximum of 5 documents. The oldest entries will be removed when the size limit is reached.

```
// Create a capped collection named "log" with a size & limit

db.createCollection("log", {
  capped: true,
  size: 51200,        // max collection size 50KB
  max: 5,             // max number of document 5
});
--------------------------------------------------------------------------------
// Insert some log entries into the capped collection
db.log.insertMany([
  { message: "Log entry 1" },
  { message: "Log entry 2" },
  { message: "Log entry 3" },
  { message: "Log entry 4" },
  { message: "Log entry 5" },
]);
--------------------------------------------------------------------------------
// Insert a new log entry to trigger the capped collection behavior
// This will remove the oldest entry (Log entry 1) to make space for the new entry

db.log.insertOne({ message: "Log entry 6" });

// Verify the contents of the capped collection
db.log.find();

// To see the most recent log entries, we can sort by natural order
db.log.find().sort({ $natural: -1 });
```

Is Capped Method : isCapped

This Method use the check collection is Capped collection or no-capped collection

```
db.log.isCapped(); // Check if the collection is capped
```

Convert To Capped Collection : convertToCapped

This Operator use to convert an existing collection to capped collection, Note : max option is not supported in converToCapped

```
// Convert an existing collection to a capped collection
db.runCommand({ convertToCapped: "students", size: 51200 });
```

Capped Max Operators : cappedMax

Modify the capped collection to change the maximum number of documents

```
//This command will not change the size of the capped collection, only the max number
db.runCommand({ collMod: "log", cappedMax: 7 });
```

Capped Size Operators : cappedSize

Modify the capped collection to change the size limit,

```
//This command will not change the maximum number of documents, only the size limit
db.runCommand({ collMod: "log", cappedSize: 7 });
```

Collection : For Testing Query

Create a Collection name students for testing Index Methods & Operators

```
db.students.insertMany([
  { _id: 1, name: "Akshay Kumar", age: 23, class: "BCA", email: "akshay@email.com"},
  { _id: 2, name: "Salman Khan", age: 24, class: "Btech", email: "salman@email.com"},
  { _id: 3, name: "Shahid Kapoor", age: 20, class: "BSc", email: "shahid@email.com"},
  { _id: 4, name: "John Abraham", age: 19, class: "BCA", email: "john@email.com"},
  { _id: 5, name: "Amir Khan", age: 24, class: "Btech", email: "amir@email.com"},
  { _id: 6, name: "Suniel Shetty", age: 22, class: "BCA", email: "suniel@email.com"},
  { _id: 7, name: "Kartik Aryan", age: 20, class: "Btech", email: "kartik@email.com"},
]);
```

Explaining Indexing : 4 Step

This code snippet demonstrates how to create, view, and drop indexes in a MongoDB collection named "students". It also shows how to run queries with explanations of their execution stats. there are examples of creating single field indexes, compound indexes, unique indexes, text indexes, and wildcard indexes. The code includes commands to insert documents into the collection and perform queries with explanations to analyze their performance. there are INDEXES in MongoDB, which are used to improve the performance of queries by allowing the database to quickly locate and access the data without scanning the entire collection. IXSCAN, COLLSCAN, FETCH and other terms refer to different types of index scans and query execution strategies in MongoDB.

```
// 1. Create an index
db.students.createIndex({ email: 1 });

// 2. View indexes
db.students.getIndexes();

// 3. Run and explain a query
db.students.find({ email: "kartik@email.com" }).explain("executionStats");

// 4. Drop Indexes
db.students.dropIndex("email_1");

-----------------------------------------------------------------------------------------------------------

// Checking executionStats Object
executionStats: {
  executionTimeMillis: 0,         // Time MongoDB took to run the query (in ms)
  totalKeysExamined: 1,           // Number of index entries scanned
  totalDocsExamined: 1,           // Number of documents fetched
  executionStages: {
    isCached: false,              // Whether the result came from cache
    stage: 'IXSCAN',              // Type of query scan (e.g., 'IXSCAN' for Index Scan)
    indexName: '$**_1',           // Name of index used (may vary)
    isUnique: false               // Whether the index is unique
  }
}
```

Create Index : Single field Indexes

This code creates an index on the "name" field of the "students" collection,

```
db.students.createIndex({
  name: 1, // 1 for ascending order, -1 for descending order
});
-----------------------------------------------------------------------------------------------------------
// Show Indexes
db.students.getIndexes();

// Run and Explain Query
db.students.find({ name: "Amir Khan" }).explain("executionStats");

// Drop Indexes
db.students.dropIndex("name_1");
```

Create Index : Compound Indexes

This code creates a compound index on the "class" and "age" fields of the "students" collection.

```
db.students.createIndex({
  class: 1, // 1 for ascending order, -1 for descending order
  age: -1, // -1 for descending order
});
-----------------------------------------------------------------------------------------------------------
// Show Indexes
db.students.getIndexes();

// View documents where class is "BCA"

db.students.find({ class: "BCA" });

// Explain the query execution stats
db.students.find({ class: "BCA" }).sort({ age: -1 }).explain("executionStats");
```

Create Index : Unique Indexes

This code creates a unique index on the "email" field of the "students" collection, ensuring that no two documents can have the same email address. If you try to insert a document with an email that already exists, it will through an

```
// Create Index

db.students.createIndex(
  { email: 1 }, // 1 for ascending order, -1 for descending order
  { unique: true } // Ensure uniqueness
);
-----------------------------------------------------------------------------------------------------------
// Show Indexes
db.students.getIndexes();
// Show documents with a specific email
db.students.find({ email: "suniel@email.com" });
// Explain the query execution stats
db.students.find({ email: "suniel@email.com" }).explain("executionStats");

// If you try to insert a document with an email that already exists,
it will through an error
db.students.insertOne(
{ _id: 8, name: "Katrina Kaif", age: 21, class: "BCA", email: "kartik@email.com"}
);
```

Create Index : Text Index

This code creates a text index on the "name" field of the "students" collection, allowing for text search capabilities.
It then performs a text search for the term "Khan" in the "name"

```
// Create Text Index
db.students.createIndex({ name: "text" });

// Show Indexes
db.students.getIndexes();

// Perform Text Search
db.students.find({ $text: { $search: "Khan" } });

// Explain the query execution stats
db.students.find({ $text: { $search: "Khan" } }).explain("executionStats");
```

Create Index : Wildcard Index

This code creates a wildcard index on all fields in the "students" collection, allowing for

```
// Create Wildcard Index
db.students.createIndex({ "$**": 1 });

// Show Indexes
db.students.getIndexes();

// Find documents with a specific email
db.students.find({ email: "suniel@email.com" });

// Explain the query execution stats
db.students.find({ email: "suniel@email.com" }).explain("executionStats");
```

MongoDB Tools : Import Json File, Backup & Restore Database

1st Step : Download & Install

Visit and download the MongoDB Command Line Database Tools from the official MongoDB website.

```
1) https://www.mongodb.com/try/download/database-tools

2) Install the downloaded tools by just simply click on Next and Next.
```

2nd Step : Set Environment Variable

After installation, set the environment variable for the MongoDB Database Tools.

```
1) search for "Environment Variables" in the Windows search bar.

2) In the System Properties window, click on the "Environment Variables" button.

3) In the Environment Variables window, under "System variables",
   double click on "Path" Variable.

4) In the Edit Environment Variable window, click on "New" and add the path to the
   MongoDB Database Tools installation directory.

5) Click OK to close all windows.

The default installation path is usually "C:\Program Files\MongoDB\Tools\100\bin".
```

Import Json Data : mongoimport

Open a new Command Prompt and execute the following command to insert data from a JSON file into a MongoDB database.

```
// Make sure to replace "D:/test.json" with the actual path to your JSON
// file, "school" with your database name, and "testing" with your collection name.

mongoimport "D:/test.json" -d school -c testing --jsonArray

db.testing.find() // verify the import by run this command
```

Backup Database & Collection : mongodump

Those commands are used to backup a MongoDB database, collection, or all databases. All command have two variant

```
// Database Backup Command : Backup the 'school' database to 'c:\backup'
mongodump -d school -o c:\backup

// Collection Backup Command : Backup the 'students' collection from the 'school' database to 'c:\backup'
mongodump -d school -c students -o c:\backup

// All Databases Backup Command
mongodump -o c:\backup // Backup all databases to 'c:\backup‘
```

Restore Database & Collection : mongorestore

those commands are used to restore the database or collection from the backup created above

```
// Restore Collection Command : Restore the 'students' collection from the backup
mongorestore -d school -c students C:\backup\school\students.bson

// Restore Database Command : Restore the 'school' database from the backup
mongorestore -d office C:\backup\school

// Restore All Databases Command : Restore all databases from the backup
mongorestore --dir C:\backup
```

Roles : Database-Specific Roles

Role	Description
read	Allows the user to read data from the database.
readWrite	Allows the user to read and write data to the database.
dbAdmin	Allows the user to perform administrative tasks (e.g., indexing, schema operations) on the database.
userAdmin	Allows the user to manage users and roles within the database.
dbOwner	Combines readWrite, dbAdmin, and userAdmin roles for the database.

Roles : Backup & Restoration Roles

Role	Description
Backup	Allows the user to back up the database
Restore	Allows the user to restore data from backups.

Roles : User Administration Roles

Role	Description
userAdminAnyDatabase	Allows the user to manage users on any database.
dbAdminAnyDatabase	Provides dbAdmin privileges for all databases.

Roles : Super user Roles

Role	Description
root	Full administrative access to all databases, users and operations, (Superuser Role)

Roles : Read Only Roles

Role	Description
readAnyDatabase	Allows read-only access to all databases.

Roles : Cluster-Level Roles

Role	Description
clusterAdmin	Provides full control over the cluster (sharding, replication, etc.).
clusterManager	Allows monitoring and managing the cluster, but without full admin privileges.
clusterMonitor	Provides read-only access to cluster status and metrics.
hostManager	Allows managing server instances and monitoring processes.

1st Step : Open configuration file

Open the MongoDB configuration file, The path may vary based on your installation, but it is typically found at:

```
C:\Program Files\MongoDB\Server\<version>\bin\mongod.cfg
```

2nd Step : Enable authentication

find the security and remove # to uncomment and add the following lines

```
security:
  authorization: enabled  // give 1 or 2 space before the line
```

3rd : Restart Mongo DB services

Save the configuration file and restart the MongoDB service for changes to take effect

```
1. Press Window + R Key to open the Run dialog.
2. Type "services.msc" and press Enter.
3. Find "MongoDB Server" in the list.
4. Right-click on it and select "Restart".
```

Check : authentication is enabled or not

Open a command prompt and connect to MongoDB using the mongo shell, Run any command that requires authentication like

```
school> show collections


if you get error like
MongoServerError[Unauthorized]: not authorized on school to execute command {
listCollections: 1, filter: {}, cursor: {}, nameOnly: true, authorizedCollections:
false, lsid: { id: UUID("5e666838-b2ba-4627-9a04-82c00c79b34c") }, $db: "school" }
It’s mean authentication is correctly implement and running


If you see a list of collection without being prompted, authentication is not enabled
correctly.
```

Create User : Admin user

Create an admin user with a username and password, this admin user have all Database access

```
1. Switch to admin database
-------------------------------------------------------------------------------
 use admin
-------------------------------------------------------------------------------


2. Create an admin user with a username and password
-------------------------------------------------------------------------------
 db.createUser({
   user: "admin",                      // User name
   pwd: "admin123",                    // User name
   roles: [{role: "root",db:"admin"}] // role and Database access
 })
-------------------------------------------------------------------------------


3. Login as Admin User
-------------------------------------------------------------------------------
 db.auth("admin","admin123")
-------------------------------------------------------------------------------
```

Create User : Developer user

Create an Developer user, Developer user can only ready school database

```
1. Switch to admin database
-------------------------------------------------------------------------------
 use admin
-------------------------------------------------------------------------------


2. Login as Admin User
-------------------------------------------------------------------------------
 db.auth("admin","admin123")
-------------------------------------------------------------------------------


3. Create an developer user with a username and password
-------------------------------------------------------------------------------
 db.createUser({
   user: "developer",                    // User name
   pwd: "dev123",                       // User name
   roles: [{role: "read",db:"school"}]  // role and Database access
 })
-------------------------------------------------------------------------------


4. Login as Developer User
-------------------------------------------------------------------------------
 db.auth("developer","dev123")
-------------------------------------------------------------------------------
```

Get all : User details

Get All Users Details first Login With Admin user that user they have root access

```
1. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------
 test> use admin                                    // switch to admin database
admin> db.auth("admin","admin123")                 // login as admin
-------------------------------------------------------------------------------

2. Get All Users Details
-------------------------------------------------------------------------------
 db.getUsers()                                  // this will return all users
-------------------------------------------------------------------------------

You get all users information (details) like this
-------------------------------------------------------------------------------

 {
   users: [
     {
       _id: 'admin.admin',
       userId: UUID('3037dc37-2fad-45d4-a058-e6652246b6ad'),
       user: 'admin',
       db: 'admin',
       roles: [ { role: 'root', db: 'admin' } ],
       mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
     },
     {
       _id: 'admin.developer',
       userId: UUID('7a723e17-76bf-4831-bf93-2ba7e0197ab0'),
       user: 'developer',
       db: 'admin',
       roles: [ { role: 'read', db: 'school' } ],
       mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
     }
   ],
   ok: 1
 }
-------------------------------------------------------------------------------
```

Get Single : User details

Get Single User Details, first Login With Admin user that user they have root access

```
1. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------
 db.auth("admin","admin123")
-------------------------------------------------------------------------------


2. Get Single User Details
-------------------------------------------------------------------------------
 db.getUser("developer")                  // Pass user name as parameter
-------------------------------------------------------------------------------

{
  _id: 'admin.developer',
  userId: UUID('7a723e17-76bf-4831-bf93-2ba7e0197ab0'),
  user: 'developer',
  db: 'admin',
  roles: [ { role: 'read', db: 'school' } ],
  mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
}
```

Update : User Details

To update user details ---> 1 switch to admin database, then login as admin, then, run update user command

```
1. Switch to admin database
-------------------------------------------------------------------------------

 use admin
-------------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------

 db.auth("admin","admin123")

-------------------------------------------------------------------------------


3. Update user role & password
-------------------------------------------------------------------------------

 db.updateUser(
     "developer",          // user name
      {pwd: "dev1234"},    // new password
     {roles: [{ role: "readWrite", db: "school" }]}
 )
-------------------------------------------------------------------------------
```

Change : User Password

A dedicated shorthand command for changing passwords

```
1. Switch to admin database
-------------------------------------------------------------------------------

 use admin
-------------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------

 db.auth("admin","admin123")

-------------------------------------------------------------------------------


3. Update user password
-------------------------------------------------------------------------------

 db.changeUserPassword(
   "developer",            // user name
   "pass123"               // new password
 )
-------------------------------------------------------------------------------
```

Drop : Single User

To drop single user ---> 1 switch to admin database, then login as admin, then, run drop command

```
1. Switch to admin database
-----------------------------------------------------------------------------

 use admin

-----------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-----------------------------------------------------------------------------

 db.auth("admin","admin123")

-----------------------------------------------------------------------------


3. Drop / delete single user
-----------------------------------------------------------------------------

 db.dropUser("developer")

-----------------------------------------------------------------------------
```

Drop : All User

To drop all user ---> 1 switch to admin database, then login as admin, then, run drop all users command

```
1. Switch to admin database
-----------------------------------------------------------------------------

 use admin

-----------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-----------------------------------------------------------------------------

 db.auth("admin","admin123")

-----------------------------------------------------------------------------


3. Drop / delete all users
-----------------------------------------------------------------------------

 db.dropAllUsers()

-----------------------------------------------------------------------------
```

Grant Role : to User

This command is used to grant additional roles to an existing MongoDB user without removing their current roles.

```
1. Switch to admin database
-------------------------------------------------------------------------------

 use admin
-------------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------

 db.auth("admin","admin123")

-------------------------------------------------------------------------------


3. Grant / (add) new roles to users
-------------------------------------------------------------------------------

 db.grantRolesToUser(
   "developer",                          // user name
   [{role: "dbOwner", db: "school"}]     // new roles
 )
-------------------------------------------------------------------------------
```

Delete : User Roles

To drop all user ---> 1 switch to admin database, then login as admin, then, run drop all users command

```
1. Switch to admin database
-------------------------------------------------------------------------------

 use admin
-------------------------------------------------------------------------------


2. Login as Admin User or that user they have root access
-------------------------------------------------------------------------------

 db.auth("admin","admin123")

-------------------------------------------------------------------------------


3. Drop / delete all users
-------------------------------------------------------------------------------

 db.revokeRolesFromUser(
   "developer",                          // user name
   [{role: "read", db: "school"}]        // roles you want to delete
 )
-------------------------------------------------------------------------------
```